



ADIGRAT UNIVERSITY
COLLEGE OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE
PROJECT ON
ADIGRAT HOSPITAL MANAGEMENT SYSTEM (AHMS) FOR ADIGRAT
HOSPITAL
SUBMITTED
IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE DEGREE OF
BACHELOR OF SCIENCE IN COMPUTER SCIENCE

PROJECT GROUP NAME

ID NO.

- | | |
|------------------------------|---------------------|
| 1. <i>HAFTOM GOITOM</i> | <i>RNS/15605/12</i> |
| 2. <i>GOYTOM EYASU</i> | <i>RNS/15570/12</i> |
| 3. <i>SENAIT MISGNA</i> | <i>RNS/19137/12</i> |
| 4. <i>KIBREMARIAM GRIMAY</i> | <i>RNS/19134/12</i> |
| 5. <i>NETSANET HAILEZGI</i> | <i>RNS/19046/12</i> |
| 6. <i>DANIEL GEBREMEDHN</i> | <i>RNS/19523/12</i> |

ADVISOR: Ins. Gereziher G.

ADIGRAT, ETHIOPIA

January 2026

<i>NAME</i>	<i>ID</i>
<i>HAFTOM GOITOM</i>	<i>RNS/15605/12</i>
<i>GOYTOM EYASU</i>	<i>RNS/15570/12</i>
<i>SENAIT MISGNA</i>	<i>RNS/19137/12</i>
<i>KIBREMARIAM GRIMAY</i>	<i>RNS/19134/12</i>
<i>NETSANET HAILEZGI</i>	<i>RNS/19046/12</i>
<i>DANIEL GEBREMEDHN</i>	<i>RNS/19523/12</i>

A Project Submitted To College Of Engineering And Technology Of Adigrat
University In Partial Fulfillment Of The Requirements For The Degree Of
Bachelor Of Science In Computer Science

Advisor Name: Ins. Gereziher G.

May 2026

Adigrat, Ethiopia

DECLARATION

Name	Signature
<i>HAFTOM GOITOM</i>	_____
<i>GOYTOM EYASU</i>	_____
<i>SENAIT MISGNA</i>	_____
<i>KIBREMARIAM GRIMAY</i>	_____
<i>NETSANET HAILEZGI</i>	_____
<i>DANIEL GEBREMEDHN</i>	_____

College: Engineering and Technology

Program: Computer Science

Project Subject: ADIGRAT HOSPITAL MANAGEMENT SYSTEM FOR ADIGRAT HOSPITAL (AHMS)

We certify that this project satisfies all the requirements as a project for the degree of Bachelor of Science.

Name of Department Head

Signature

This is to certify that I have read this project and that in my opinion it is fully adequate, in scope and quality, as a project for the degree of Bachelor of Science.

Name of Advisor

Signature

Examining committee members

Signature

Date

1. Chairman _____

2. Examiner 1 _____

3. Examiner 2 _____

It is approved that this project has been written in compliance with the formatting rules laid down by the college of the university.

ACKNOWLEDGMENT

First and foremost, we give our sincere thanks to the Almighty God for His guidance, strength, wisdom, and protection throughout our academic journey and during the successful completion of this final year project. Without His blessings, this achievement would not have been possible.

We would like to express our deepest gratitude to **Adigrat University**, particularly the **Department of Computer Science**, for providing us with the academic foundation and conducive learning environment necessary to undertake this project. This **AHMS** has enabled us to integrate theoretical knowledge with practical system development skills.

We are highly grateful to our project advisor Ins. Gerezher G. for his continuous guidance, constructive feedback, encouragement, and professional support throughout the analysis and design of this project. His technical insights and academic supervision played a crucial role in the successful completion of this project.

Our sincere appreciation also goes to the Hospital management and medical professionals who generously shared their time, experience, and domain knowledge during data collection. In particular, we would like to thank **Dr. Gaym**, **Dr. Alem**, and as well as the nurses and administrative staff, for their valuable cooperation, explanations, and practical insights, which were essential in understanding real-world healthcare workflows.

Furthermore, we extend our thanks to all individuals who supported us directly or indirectly by providing information, ideas, and constructive suggestions during the development of this project.

Finally, we acknowledge the cooperation and teamwork among all group members. The collective effort, collaboration, and mutual support within the group were key factors in the successful completion of this final year project.

ABSTRACT

In many healthcare institutions, including Adigrat Hospital, hospital managements were traditionally managed using manual, paper-based systems. This approach caused challenges such as loss and misplacement of records, slow retrieval of patient information, data inconsistency, increased administrative workload and risks to patient data confidentiality. These issues negatively affected the efficiency and quality of healthcare service delivery. This project presents the design and development of a web-based AHMS for Adigrat Hospital to address the limitations of the existing manual system. The proposed system enabled secure digital storage, retrieval, and management of patient health information, including patient registration, diagnosis record, treatment records, appointment scheduling, and report generation. Role-based access is implemented for system users such as administrators, doctors, nurses, receptionists, pharmacists, and laboratory technicians to ensure data security and proper system usage. The system is developed using an incremental software development methodology, supported by requirement analysis techniques such as interviews, observations, and document reviews. Object-oriented modeling tools including use case diagrams, sequence diagrams, activity diagrams, and class diagrams were used during system design. The implementation of the AHMS improved efficiency, accuracy, accessibility, and security of patient records while reducing paper usage and administrative costs.

The AHMS for Adigrat Hospital was developed to modernize patient health information management by transitioning from manual, paper-based processes to a centralized digital platform. Designed using a web-based client-server architecture, the system provides a secure, efficient, and scalable solution that allows authorized hospital staff including doctors, nurses, laboratory technicians, and administrators to access accurate patient data in real time. Key design features focus on data integrity, role-based access control, and user-friendly interfaces, all aimed at minimizing human error, enhancing workflow efficiency, and ensuring the confidentiality of sensitive medical records. This project provides a robust foundation for improved hospital operations, offering a scalable framework that supports future expansion and departmental integration.

TABLE OF CONTENTS

LIST OF FIGURES	V
LIST OF TABLES	VI
LIST OF SYMBOLS AND NOTATIONS	VII
CHAPTER ONE	1
INTRODUCTION	1
1.1 Background of the Organization	1
1.2 Statement of the Problem	2
1.3 Objectives of the project	2
1.3.1 General Objective	2
1.3.2 Specific Objectives	2
1.4 Scope of the project	2
1.5 Limitation of the project	3
1.6 Significance of the Project	3
1.7 Methodology	3
1.7.1 Data Gathering	3
1.7.2 System Analysis and Design Methodology	3
1.7.3 Implementation Methodology	5
1.7.4 Development Methodology	6
1.7.5 Feasibility Studies	7
1.7.6 Testing Procedures	7
1.8 Organization of the project	8
CHAPTER TWO	9
REQUIREMENT ANALYSIS DESCRIPTION	9
2.1 Overview of the Existing System	9
2.2 Player of the Existing System	9
2.3 Activities of the System	10
2.3.1 Patient Record Management	10
2.4 Business Rule of the Existing System	11
2.5 Problems of the existing system	11
2.5.1 Performance	11
2.5.2 Security and Controls	12
2.5.3 Efficiency	12
2.5.4 Input and output	12
2.6 Overview of the proposed System	13

2.7 Specific Requirements	13
2.7.1 Functional Requirements	13
2.7.2 Non Functional Requirements	14
2.8 System Requirements	15
2.8.1 Hardware Requirements	15
2.8.2 Software Requirements	15
2.9 Constraints and Assumptions	16
2.9.1 Constraints	16
2.9.2 Assumptions	16
CHAPTER THREE	17
SYSTEM MODELING	17
3.1 Introduction	17
3.2 System Use Case Model	17
3.2.1 Actor Identification	17
3.2.2 Description of Use Case	19
3.3 Sequence Diagram	24
3.4 Conceptual Class Model	28
3.5 Activity Diagram	29
CHAPTER FOUR	33
SYSTEM DESIGN	33
4.1 Introduction	33
4.2 Design Goal	33
4.3 System Architecture	34
4.3.1 Presentation Layer (Client Layer)	34
4.3.2 Application Layer (Server Layer)	35
4.3.3 Data Layer (Database Layer)	35
4.3.4 Architecture Overview	35
4.4 Class Diagram	35
4.4.1 Introduction	35
4.4.2 Main Classes of the System	36
4.4.3 Relationships between Classes	37
4.5 Deployment Diagram	37
4.6. Persistence Data Management	38
4.7 User Interface	39
CHAPTER FIVE	42
IMPLEMENTATION AND TESTING	42
5.1 Implementation Overview	42

5.1.1 Objective of the implementation	42
5.2 Sample code and overview	42
5.3 Testing Overview	48
5.3.1 Testing by Scope	48
5.3.2 Testing by Requirements (System Testing)	49
5.3.3 Test Implementation (Validation Testing)	51
5.4 Hardware and Software Acquisition	51
5.5 User Manual Preparation	52
5.6 Installation Process	53
5.7 Start-up Strategy	53
CHAPTER SIX	54
CONCLUSIONS AND RECOMMENDATIONS	54
6.1 Conclusion	54
6.2 Recommendations	54
References	56
Appendix	57

LIST OF FIGURES

Figure 3.1: Use case diagram.....	19
Figure 3.2: Sequence Diagram for Login.....	24
Figure 3.3: Sequence Diagram for Create Account.....	24
Figure 3.4: Sequence Diagram for Delete Account.....	25
Figure 3.5: Sequence Diagram for Display Medication.....	25
Figure 3.6: Sequence Diagram for Register New Patient.....	26
Figure 3.7: Sequence Diagram for Access Patient Record.....	26
Figure 3.8: Sequence Diagram for Request Lab Test.....	27
Figure 3.9: Sequence Diagram for Schedule Appointment.....	27
Figure 3.10: Sequence Diagram for Update Patient Information.....	28
Figure 3.11: Conceptual Diagram model.....	29
Figure 3.12: Activity Diagram for Login.....	30
Figure 3.13: Activity Diagram for Create Account.....	31
Figure 3.14: Activity Diagram for Access Patient Record.....	31
Figure 3.15: Activity Diagram for Register New Patients.....	32
Figure 3.16: Activity Diagram for Update New Patient.....	32
Figure 4. 1: class diagram for AHMS.....	36
Figure 4. 2: Deployment Diagram of AHMS.....	37
Figure 4. 3: Mapping object.....	39
Figure 4. 4: Home Page.....	40
Figure 4. 5: system admin page.....	40
Figure 4. 6: receptionist page.....	41
Figure 6.1: Vision and mission of the hospital.....	57
Figure 6.2: Organization structure.....	57
Figure 6.3: Receptionist room of the hospital.....	58

LIST OF TABLES

Table 2.1: functional requirements.....	13
Table 2.2: non-functional requirements.....	14
Table 3.1: create account use case description.....	19
Table 3.2: login use case description.....	20
Table 3.3: delete account use case description.....	20
Table 3.4: access patient record use case description.....	21
Table: 3.5: update patient information use case description.....	21
Table 3.6: request lab test use case description.....	22
Table 3.7: enter lab test result use case description.....	22
Table 3.8: display medication use case description.....	22
Table 3.9: register new patient use case description.....	23
Table 3.10: schedule appointment use case description.....	23
Table 5.1: Login Module.....	49

LIST OF SYMBOLS AND NOTATIONS

AHMS – Adigrat Hospital Management System

CSS - Cascading style sheet

DBMS - Database Management System

DFD – Data Flow Diagram

ERD – Entity Relation Ship Diagram

HTML - Hyper Text Markup Language

HTTP - Hyper Text Transfer Protocol

IP - Internet Protocol

LAN - Local Area Network

PHP - Hypertext Preprocessor

RAM – Random Access Memory

RJ-45 - Registered Jack-45

SQL - Structured Query Language

UI - user interface

UML – Unified Modeling Language

URL – Uniform Resources Locator

CHAPTER ONE

INTRODUCTION

In the modern era of digital transformation, technology plays a vital role in enhancing healthcare services and efficiency. AHMS for Adigrat Hospital aims to digitalize the traditional manual hospital management process by introducing a secure, simple, and efficient digital platform that allows patients to record their data digitally. This system eliminates many challenges of manual hospital management such as data losses, human error in storing record, and appointment issues by providing a reliable and user-friendly digital solution.

1.1 Background of the Organization

Adigrat General Hospital is a governmental institution established under Tigray bureau of health. It is located in the eastern part of Tigray, Adigrat city.

1.1.1 Vision

By following a modern financial and human resource management executive system and establishing a framework for good governance, to fulfill the Comprehensive Specialized Hospital Strategic Plan through knowledge, skill, and attitude; to ensure it remains a leading and competitive resource; and to see the hospital become a center where clients receive fast and quality medical services.

1.1.2 Mission

To fulfill the missions of higher education institutions, which are teaching, research, and community service, by producing a workforce that is competent and competitive in knowledge, skill, and attitude; to work toward the establishment of a modern operational system so that Comprehensive Specialized Hospitals thrive; and additionally, to provide support and follow-up for the delivery of quality and equitable medical services.

1.1.3 Institutional Values

- Learning by doing
- Inclusivity
- Honesty
- Accountability
- Team spirit
- Community-wide activity and responsibility
- Fighting corruption

- Living in a process of change
- We care for HIV/AIDS together

1.2 Statement of the Problem

Currently, the hospital management process at Adigrat Hospital is largely manual and paper-based, leading to several challenges such as misplaced patient files, slow retrieval of medical information, difficulty maintaining complete patient history, high cost, and high administrative workload. The manual system also creates long queues as patients wait for their files to be located and updated. These limitations reduce the quality of healthcare service, cause delays in treatment, and increase the risk of medical errors. Therefore, there is a need for a secure, digital, and efficient health record system that simplifies patient data management while ensuring accuracy, privacy, and reliability.

1.3 Objectives of the project

1.3.1 General Objective

The general objective of this project is to develop a Web-based AHMS for Adigrat Hospital.

1.3.2 Specific Objectives

The specific objectives of this project are:

- To enable patients to register and record using hospital ID.
- To enable updating and retrieval of patient information.
- To enable the system to generate reports.
- To enable appointment scheduling and sending treatments.
- To enable the doctor and nurse to access patient information in time interval.

1.4 Scope of the project

The AHMS for Adigrat Hospital is designed to digitalize patient health information management and related hospital activities. The system covers patient registration, diagnosis and treatment recording, appointment scheduling, follow-up management, and report generation.

The system supports different user roles including administrators, doctors, nurses, receptionists, pharmacists, and laboratory technicians. However, the system does not include financial management, billing, and inventory management or automated medical decision support.

1.5 Limitation of the project

Although the proposed system addresses many challenges of the existing manual system, it has certain limitations. The system does not include financial and billing functionalities. It does not provide automated diagnosis or treatment recommendations.

Additionally, the system does not manage hospital resources such as equipment or staff scheduling. Resistance to change from staff accustomed to manual systems may also affect adoption during early implementation.

1.6 Significance of the Project

After implementing this system it is very easy to hospital management and accessing information of the patients, reduces cost for papers on each part.

The system ensures:

- **Efficiency:** Faster health record and generating report.
- **Accessibility:** Enables fast and simple access of patient data.
- **Transparency:** Reduces human interference and ensures fair results.
- **Security:** Protects patient data and prevents multiple health information .
- **Cost Reduction:** Decreases expenses related to papers and copies.

1.7 Methodology

1.7.1 Data Gathering

Observation: As we observed the patients record and as a consumers in the hospital they use manual system completely.

Interview:

We used interviews to collect information about the existing workflow of patient data management, including how patient information is recorded, updated, stored, and retrieved.

The interviews helped us understand the tasks performed by medical staff during patient registration, treatment documentation, and record handling. Through these discussions, we identified the challenges in the current manual system and gathered insights necessary for designing an efficient AHMS.

1.7.2 System Analysis and Design Methodology

The development of the AHMS follows a structured and systematic methodology to ensure that the system meets the functional and non-functional requirements of Adigrat Hospital. The system analysis and design process involves understanding the existing manual hospital management

system, identifying its limitations, and proposing a digital solution that improves efficiency, accuracy, and data security.

To achieve this, the methodology includes the following steps:

1. Requirement Analysis

During this phase, detailed information was collected from hospital staff, including receptionist, nurses, doctors, and administrative personnel. The objective was to understand how patient records are currently managed, what challenges exist, and what functionalities are needed in the new system. Techniques such as interviews and observations were used.

2. System Modeling

Based on the gathered information, system models were prepared to describe the workflow and data flow of the new system. This included use case diagrams, data flow diagrams (DFDs), entity-relationship diagrams (ERDs), and system architecture diagrams. These models help visualize user interactions, data storage, and system processes.

3. System Design

In the design phase, the proposed system's structure was defined. This includes:

- **Interface Design:** Designing user-friendly interfaces for admins, doctors, nurses, and receptionists.
- **Database Design:** Creating a well-structured database to store patient, doctor, nurse and others records securely.
- **Process Design:** Defining how data will be processed, updated, and retrieved.
- **Security Design:** Establishing authentication, authorization, backup, and encryption mechanisms.

4. Feasibility Review

The analysis and design approach also includes reviewing the technical, operational, and economic feasibility of the proposed system to ensure it can be implemented successfully in the hospital environment.

This methodology ensures that the AHMS is developed in a way that aligns with the needs of the hospital, supports efficient healthcare service delivery, and provides a reliable and secure system for managing patient records.

1.7.3 Implementation Methodology

The implementation of the AHMS follows a systematic and controlled approach to ensure that the system is successfully deployed and integrated into the workflow of Adigrat Hospital. The methodology focuses on preparing the environment, installing system components, training users, and ensuring smooth operation.

The implementation process includes the following major steps:

1. System Installation

In this stage, the necessary software and hardware components required to run the AHMS are installed. This includes setting up servers, installing the database system, configuring network connections, and deploying the application modules. The installation ensures that all system components operate correctly in the hospital environment.

2. Data Migration

Existing patient records and other manual tasks which are stored in paper-based files, need to be transferred into the digital system. This process involves manually entering patient information or scanning documents to ensure accurate data transfer. Care is taken to maintain data integrity and avoid duplication or loss during migration.

3. User Training

Proper training is given to health professionals such as receptionist, nurses, doctors, and administrative staff. The training covers how to log in, view patient records, update treatment information, schedule appointments, and use other system features. Training sessions help staff become confident and reduce resistance to adopting the new digital system.

4. Pilot Implementation

Before full-scale deployment, a pilot version of the system is implemented in selected departments such as the outpatient department or registration unit. This helps evaluate system performance in real hospital conditions and gather feedback from staff for improvements.

5. Full System Deployment

After testing the pilot version and making necessary adjustments, the system is fully deployed across all relevant hospital departments. At this stage, all users begin using the system for daily operations, and the manual record system is gradually phased out.

6. Support and Maintenance

Technical support is provided during and after deployment to assist users with any issues they encounter. Regular maintenance activities include updating the system, fixing bugs, monitoring server performance, and ensuring data security.

This structured implementation methodology ensures that the AHMS is adopted smoothly, minimizes disruption to hospital operations, and provides reliable digital access to patient information.

1.7.4 Development Methodology

The development of the AHMS follows a systematic software development approach to ensure that the system is reliable, scalable, secure, and user-friendly.

For this project, the Incremental Software Development Methodology was chosen because it allows the system to be built, tested, and improved in phases. This approach is suitable for hospital environments where continuous feedback and gradual deployment are essential.

The development process includes the following stages:

1) Planning Phase

In this stage, the overall project scope, objectives, and system requirements were identified.

Planning also involved determining the resources, tools, and technologies required for system development.

2) Analysis Phase

The requirements gathered from hospital staff were analyzed to understand functional and non-functional needs. Gaps in the existing manual management system were evaluated to determine what features should be included in each development increment.

3) Design Phase

The system architecture, user interface, database structure, and security mechanisms were designed before coding. This includes designing screens for patient registration, medical history management, appointment handling, and user authentication.

4) Development Phase

The system was developed incrementally. Each module such as patient registration, record updating, appointment scheduling, and staff management was coded, tested, and refined before integrating with other modules. This ensured early detection of errors and allowed continuous improvement.

5) Testing Phase

After each increment, unit testing, integration testing, and system testing were performed to verify that the module functions correctly and integrates smoothly with other components.

Feedback from medical staff was used to improve usability.

6) Deployment Phase

After successful testing, the system is prepared for deployment within the hospital environment. Deployment may initially occur in small sections (ex. outpatient department) before full-scale implementation.

7) Maintenance Phase

The maintenance stage involves updating features, fixing bugs, improving security, and ensuring the system remains fully functional as hospital needs evolve.

Using the incremental methodology ensures that the AHMS is developed efficiently, with continuous user feedback, minimal risks, and higher system quality.

1.7.5 Feasibility Studies

Feasibility is the measure of how the new system is viable or beneficial from the existing system. It is an ongoing process done frequently during system development projects in order to achieve a creeping commitment from the user and to continually assess the current status of the project. (1)

1.7.5.1 Technical feasibility

Usually new systems established in order to overcome the technical illness of the previous system. In the same way this system is technically capable to be applied easily to the problem identified in the existing system. (1)

1.7.5.2 Operational feasibility

Operational feasibility examines whether the system will work effectively in the hospital environment and whether the users will accept it.

The AHMS is operationally feasible because:

- It simplifies daily tasks for receptionists, nurses, and doctors by making patient information easy to access and update.
- It reduces long queues caused by searching for paper files.
- Staffs are willing to adopt the new system after proper training since it improves workflow efficiency.
- The system improves accuracy of patient records, reduces errors, and enhances service delivery.
- Hospital management supports the modernization effort.

1.7.6 Testing Procedures

Before directly deploying this system the team performed different types of testing procedures for its functionality. The system tested by the following testing types. (1)

1.7.6.1 Unit Testing

We tested each process independently to check their functionality.

1.7.6.2 Integration Testing

After we tested each unit of the system we performed an integration test to check whether the system meets all the functional requirements. When a number of components are complete we tested the components to ensure that they integrate well with each other, the operating system, and other components.

1.7.6.3 System Testing

Testing conducted on a complete, integrated system to evaluate the system performance with its specified requirements. System Testing falls within the scope of black box testing and should require no knowledge of the inner design of the code or logic. Series of reliability and functionality tests designed to find software engineering errors.

1.8 Organization of the project

This project report is organized into three main chapters, each focusing on a specific aspect of the AHMS for Adigrat Hospital.

Chapter One: Introduction

This chapter provides an overall introduction to the project. It includes the background of Adigrat Hospital, the problem statement, objectives (general and specific), scope, limitations, and significance of the project. It also describes the methodology used for data collection, system analysis, system design, development, implementation, and feasibility studies.

Chapter Two: Requirement Analysis Description

This chapter discusses the analysis of the existing manual hospital management system used by the hospital and identifies its major problems and limitations. It describes the players (actors) involved in the system, their activities, and the business rules. The chapter also presents an overview of the proposed Adigrat Hospital Management System, including functional and non-functional requirements, system requirements (hardware and software), constraints, and assumptions.

Chapter Three: System Modeling

This chapter focuses on the design and modeling of the proposed system using object-oriented techniques. It includes system use case diagrams, detailed use case descriptions, sequence diagrams, activity diagrams, and the conceptual class diagram. These models illustrate how different actors interact with the system and how system components work together to achieve the intended functionality.

CHAPTER TWO

REQUIREMENT ANALYSIS DESCRIPTION

2.1 Overview of the Existing System

Currently, Adigrat Hospital uses a **traditional, paper-based hospital management system**, which is time-consuming, prone to errors, and difficult to manage as patient volume increases. In the existing manual process, almost every activity from patient registration to record retrieval, diagnosis documentation, prescription handling, and reporting is completed on paper.

Hospital staff such as registration clerks, nurses, physicians, pharmacists, and record officers must manually record patient information, search for physical files, update patient histories, prepare paper cards, and store or retrieve records from shelves. These tasks require significant manpower, depend on staff experience, and often lead to delays in providing timely healthcare services.

The manual record-keeping method is also vulnerable to **data loss, duplication and misplacement of patient files, illegible handwriting, unauthorized access, and delays in accessing vital medical information**. Because of the paper-based workflow, generating reports, tracking patient history, sharing information between departments, and maintaining continuity of care becomes difficult.

To address these limitations, **AHMS** is a more efficient and reliable solution. It reduces human effort, minimizes operational costs, improves accuracy, enhances patient data security, enables faster record retrieval, and supports better decision-making by providing real-time access to complete patient medical histories.

1.1 Player of the Existing System

An existing system comprises different players to carry out its job. Among those different actors (players), the most common are:

- **System administrator:**

- ✧ Oversees hospital operations, ensures proper maintenance of patient records, manages user accounts (doctors, nurses, and staff), and monitors hospital workflows.
- ✧ Handles reporting, audits, and ensures that hospital processes comply with regulations.

- **Staff administrator:**

- ✧ Includes registration clerks, record officers, and ward staff who manage patient registration, appointments, and maintenance of medical records.
- ✧ Responsible for organizing physical patient files, updating patient information, and coordinating between departments.
- **Doctor/Nurse:**
 - ✧ Includes physicians, nurses, and other medical staff who provide care to patients.
 - ✧ Accesses patient medical history, records diagnoses, prescribes medications, and updates treatment information.
- **Patient:**
 - ✧ The primary user of the hospital services.
 - ✧ Provides personal and medical information, visits doctors, and can give feedback or complaints regarding hospital services.

1.2 Activities of the System

1.2.1 Patient information Management

The management of patient information involves multiple steps and actors. The workflow is as follows:

- **Doctor/Department Head:**
 - ✧ Prepares patient treatment plans, medical instructions, and care schedules based on patient conditions.
- **Hospital Administrative Staff (Receptionists/Record Officers):**
 - ✧ Organize and maintain patient files, including updating medical histories, lab results, and treatment schedules.
 - ✧ Distribute physical copies of patient records or summaries to the respective departments, doctors, and wards.
- **Department Heads / Senior Doctors:**
 - ✧ Review patient treatment schedules to prevent overlaps in appointments, medication administration, or diagnostic tests.
 - ✧ Adjust schedules according to doctor availability, ward capacity, and urgency of patient care.
- **Patients:**

- ✧ May receive copies of their appointment schedules or treatment plans.
- ✧ In some cases, patient representatives are given information for coordination of care.

1.3 Business Rule of the Existing System

- **Unique Patient Records:** Each patient must have a unique medical record number; no two patients share the same ID.
- **Single Appointment per Time Slot:** A patient cannot have multiple appointments with the same doctor at the same time.
- **Authorized Access Only:** Only authorized staff (doctors, nurses, administrative personnel) can access patient files.
- **Sequential Record Updates:** Patient medical records must be updated in chronological order to ensure accurate medical history.
- **Confidentiality:** Patient information must be kept confidential; unauthorized sharing is prohibited.

1.4 Problems of the existing system

The current **paper-based hospital management system** faces numerous challenges. These problems appear across several operational areas such as **performance, security, efficiency, cost, and human resource utilization**. The following points describe the major limitations observed in the existing manual hospital management system process.

1.4.1 *Performance*

- **Prone to error**

Since patient information is recorded manually on paper, the system is highly vulnerable to mistakes such as incorrect entries, illegible handwriting, misplaced files, and inconsistencies across departments. These errors directly affect the quality of healthcare delivery.

- **Time consuming**

the manual system consumes more time physically search shelves, locate files, update documents, and return them to storage, causing delays in patient service, diagnosis, treatment, and reporting.

1.4.2 Security and Controls

The existing paper-based hospital management system provides weak security and poor control mechanisms. Sensitive patient information may be exposed or lost because physical files are easily accessible, damaged, or misplaced.

1.4.3 Efficiency

Because manual operations require extensive use of physical resources, the existing system suffers from low efficiency:

- Large amounts of paper and storage space are required to keep medical records.
- Staff spends significant time searching, updating, and organizing patient files.
- Delays in accessing information slow down treatment decisions and patient flow.
- Duplicate tests may occur because past medical history is not easily accessible.

1.4.4 Input and output

The input of information is often redundant, inaccurate, disorganized, and time-consuming. Patient data such as personal details, medical history, diagnoses, test results, and treatment plans are recorded manually on paper forms.

➤ Problems with Input:

- ✧ Manual entry may lead to duplication of records or missing information.
- ✧ Handwritten records can be illegible or inconsistent, making it difficult for staff to interpret correctly.
- ✧ Staff must spend extra time organizing and filing paper records, which increases workload and administrative overhead.

➤ Problems with Output:

- ✧ Reports, summaries, or patient histories generated from manual records are often inaccurate or delayed.
- ✧ Retrieving patient information requires physically searching files, which is time-consuming and prone to errors.
- ✧ Consolidated reports for management, audits, or research are difficult to produce because data is scattered across multiple paper files.

1.5 Overview of the proposed System

Based on our Observations and interviews with hospital staff and users, we identified several challenges in the existing paper-based hospital management system. To address these issues, we propose the development of a web-based AHMS. This system is designed to overcome the limitations of the current manual process and improve hospital operations significantly.

The proposed system will provide the following key advantages and features:

- Accuracy in Data Handling.
- Fast Operation and Quick Response Time.
- Flexibility and Accessibility.
- Data Backup and Recovery.
- User-Friendly Interface.

1.6 Specific Requirements

1.6.1 Functional Requirements

The functional requirements of the proposed **AHMS** describe the major functionalities expected from the system to meet the objectives of the project. The system must provide the following features:

The priority for all requirement description in the below table is High.

Table 2.1: functional requirements

Req. ID	User Type	Requirement Description
01	System administrator	The system shall allow the system administrator to Add ordered user accounts (doctors, nurses, and staff).
02	Staff Administrator	The system shall allow the staff administrator to search/view active, discharged, and under-treatment patient records.
03	Administrator	The system shall allow the administrator to login using username and password and it allows changing their password.
04	Administrator	The system shall allow the administrators to logout.

Req. ID	User Type	Requirement Description
05	System administrator	The system shall allow the system administrator to delete ordered user accounts (doctors, nurses, and staff).
06	Staff administrator	The system shall allow the staff administrator to view reports.
07	Staff administrator	The system shall allow the staff administrator to post announcements regarding hospital activities or health campaigns.
08	Doctor/Nurse	The system shall allow the doctor/nurse to verify patient identity by entering patient ID, when receives orders from receptionist.
09	Doctor/Nurse	The system shall allow the doctor/nurse to view announcements posted by staff administrator.
10	Doctor/Nurse	The system shall allow the doctor/nurse to login using username and password and it allows changing their password.
11	Doctor/Nurse	The system shall allow the doctor/nurse to logout.

1.6.2 Non Functional Requirements

Non-functional requirements describe **system qualities and constraints** that are visible to users but are not directly related to specific functionalities.

The priority for all requirement description in the below table is High.

Table 2.2: non-functional requirements

Requirement ID	Requirement Description
01	The system should provide a simple, responsive, and user-friendly interface . By designing an intuitive interface, hospital staff can access and communicate information quickly with minimal learning curve.
02	The system should support concurrent users , allowing multiple doctors, nurses, administrators, and patients to use the system simultaneously without performance degradation.
03	The system should minimize user and system errors . Error messages should be clear and guide users on how to correct mistakes, ensuring accurate patient records.

Requirement ID	Requirement Description
04	The system shall restrict access to authorized users only , preventing any user from viewing or modifying information they are not permitted to access.
05	The system shall be available 24/7 to authorized users with an internet connection, ensuring continuous access to patient records and hospital data.

1.7 System Requirements

1.7.1 Hardware Requirements

- **For Server Computer:** Minimum Hardware Requirements:
 - ✧ Processor: Intel Pentium 4, 3.0 GHz or higher
 - ✧ RAM: 4 GB or higher
 - ✧ Hard Disk: 80 GB or higher
- **For Client Computer (Doctors, Nurses, Admin Staff, and Patients):** Minimum Hardware Requirements:
 - ✧ Processor: Intel Pentium 4, 2.0 GHz or higher
 - ✧ RAM: 512 MB or higher
 - ✧ Hard Disk: 40 GB or higher

1.7.2 Software Requirements

The software requirements for installing the AHMS include:

1. **Operating System:**
 - **Server Computer:** Microsoft Windows Server 2016 or higher versions
 - **Client Computer:** Any standard operating system (Windows, Linux, or macOS)
2. **Web Server and Database:**
 - WAMP (SQL Server, PHP /Python) or PHP (latest stable version)
3. **Web Browser (Client Computer):**
 - Mozilla Firefox version 20 or higher
 - Google Chrome (latest version)

- Opera version 9 or higher
- Internet Explorer version 9 or higher

4. **Application Software (Server and Client Computer):**

- Microsoft Office Word 2007 or higher: for creating, formatting, and digitally signing letters, reports, and hospital documentation

1.8 Constraints and Assumptions

1.8.1 *Constraints*

- The proposed AHMS cannot function during a **power outage** unless backup power is available.
- Hospital staff and patients cannot access or print medical records if **printers or network devices fail**, making it difficult to retrieve or provide records on time.
- Internet connectivity issues may temporarily restrict access to the web-based system for authorized users.

1.8.2 *Assumptions*

- It is assumed that the hospital will have **trained staff** to operate and manage the system efficiently.
- Users (doctors, nurses, and administrative staff) have **basic computer literacy** to interact with the system.
- Users must know the **hospital's web application address (URL)** to access the system.
- Users are assumed to have **basic knowledge of the English /Amharic language** for navigating the system interface and reading system messages.

CHAPTER THREE

SYSTEM MODELING

2.1 Introduction

We used object-oriented system development methodology. Because the object system development approach gives easier and natural way to breakdown problems into simple, small and manageable components so that it reduces a vague appearance of the big problem. Moreover, it is the predominately used and popular method in present software development trend.

The major activities described in this chapter are constructing a use case model, documenting the use case course of events, constructing sequence and activity diagram analysis level class diagram and so on.

2.2 System Use Case Model

A use case model is a graphical representation or a diagram that describes the interactions between actors (users or external systems) and a system under consideration. It is a key component of the Unified Modeling Language (UML), which is widely used in software development to model and document system requirements. It is a model of how different types of users interact with the system to solve the problem. As such, it describes the goals of the users, the interactions between the users and the system and the required behavior of the system is satisfying these goals.

2.2.1 Actor Identification

An essential use case is complete, meaningful and well-designed from the point of view of the users in some role or roles in relation to a system and that embodies the purpose or intentions underlining the inter action.

An actor represents anything or anyone that interacts with the system.

Actors:

- | | |
|------------------------|-------------------|
| ➤ System administrator | ➤ Receptionist |
| ➤ Doctors | ➤ Pharmacist |
| ➤ Nurses | ➤ Lab Technicians |
| ➤ Staff administrator | |

Use-Cases:

- | | |
|-------------------|-------------------------|
| ✧ Assign Schedule | ✧ Access Report |
| ✧ Create Account | ✧ Access Patient Record |
| ✧ Delete Account | |

- ✧ Update Patient Information
- ✧ Request Lab Test
- ✧ Prescribe Medication
- ✧ Set Appointment
- ✧ Follow-Up Patients
- ✧ View Record
- ✧ Feedback
- ✧ Register New Patient
- ✧ Schedule Appointment
- ✧ Reset Account
- ✧ Manage Schedules
- ✧ View Prescription
- ✧ Receive command
- ✧ Send command
- ✧ Dispense Medication
- ✧ View Lab Test Request
- ✧ Change password
- ✧ Perform Lab Test
- ✧ Enter Lab Test Result

Use-case Diagram

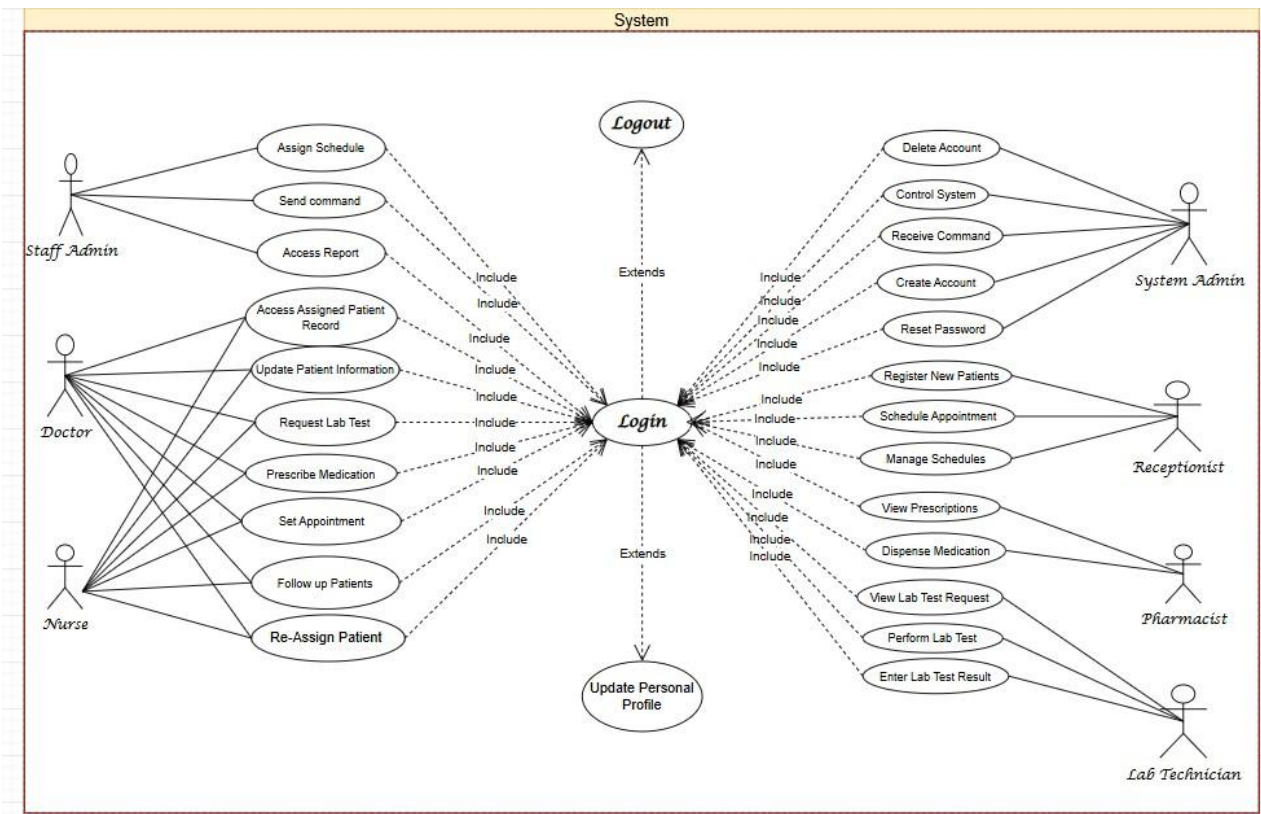


Figure 3.1: Use case diagram

2.2.2 Description of Use Case

Table 3.1: Create account use case description

Use case name		Create account
Actor		System administrator
Pre-condition		Admin is logged in
Post condition		New user account is created
Basic course of action	User action	System response
	1. Admin selects “ create account”	
		2. System displays registration form
	3. Admin enters user details	
		4. system validates data and saves account

		5. Use case ends
--	--	------------------

Table 3.2: Login use case description

Use case name		Login
Actors		Admin, doctor, Nurse, Receptionist, pharmacist, Lab Technician
Pre-condition		User has a valid account
Post condition		User is authenticated and redirected to their authorized dashboard
Basic course of action	User action	System response
	1. User opens the system homepage and clicks login	
		2. System displays login form
	3. user enters username and password	
		4. system checks validity and if valid logged in successfully 5. Use case ends
Alternative course of action A	If the username or password is not valid at step 4 of basic course of action	
	User action	System response
		4.1 system displays error message
	4.2 user views error message	
		4.3 system redisplay login form

Table 3.3 Delete account use case description

use case name		Delete account
Actor		System administrator
Pre-condition		Admin is logged in
Post condition		User account is removed
Basic course of action	User action	System response
	1.admin selects user account	
		1. System displays account detail

	2. Admin clicks delete	
		4.System confirms deletion
		5.Use case end

Table 3.4: Access patient record use case description

Use case name		access patient record
Actors		Doctor, Nurse, Receptionist
Pre-condition		Patient record exists.
Post condition		Patient record is displayed
Basic course of action	User action	System response
	1. Actor searches patient	
		2. System retrieves record
		3. System displays patient information
		4. Use case ends

Table: 3.5: Update patient information use case description

Use case name		Update patient information
Actors		Doctor , Nurse
Pre-condition		Actor logged in and patient record exists
Post condition		Patient information is updated
Basic course of action	User action	System response
	1. Actor edits patient data	
		2. system validate data
		3. System updates data
		4. Use case ends

Table 3.6: Request lab test use case description

Use case name		Request lab test
Actor		Doctor, Nurses
Pre-condition		Patient record exists
Post condition		Lab test request sent
Basic course of action	User action	System response
	1. Actors selects patient	
		2. system displays test option
	3. Actors requests test	
		4. System sends request
		5. Use case ends

Table 3.7: Enter Lab test result use case description

Use case name		Enter Lab test result
Actor		Lab technician
Pre-condition		Test performed
Post condition		Result stored in patient record
Basic course of action	User action	System response
	1. Technician enters result	
		2. System saves result
		3. Use case ends

Table 3.8: Display medication use case description

Use case name		Display medication
Actor		pharmacist
Pre-condition		Prescription exists.
Post condition		Medication information displayed
Basic course	User action	System response

of action	1. Pharmacist views prescription	
		2. System displays medication
		3. Use case ends

Table 3.9: Register new patient use case description

Use case name		Register new patient
Actor		receptionist
Pre-condition		Receptionist logged in
Post condition		Patient registered
Basic course of action	User action	System response
	1. Receptionist enters patient data	
		2. System validates
		3. System saves patient
		4. Use case ends

Table 3.10: Schedule appointment use case description

Use case name		schedule appointment
Actor		Receptionist, Doctor, Nurses
Pre-condition		Actor logged in and patient exists
Post condition		Appointment scheduled
Basic course of action	User action	System response
	1. Actor selects date and time	
		2. System checks availability
		3. System saves appointments
		4. Use case ends

3.3 Sequence Diagram

The Sequence diagram is used primarily to show the interaction between objects in the sequential order. Sequence diagrams are usually created to show the flow of message among objects and actors in the system.

In this section we are going to discuss about the sequence of an activity from one step to other step to clearly define how its flow through the system.

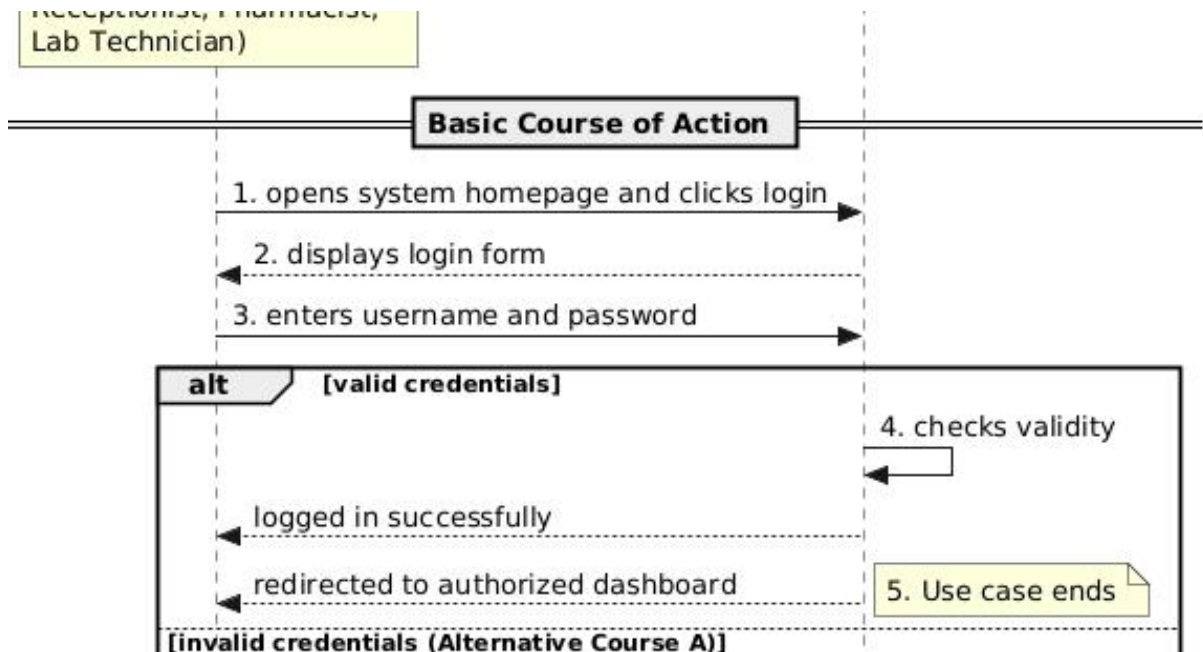


Figure 3.2: Sequence Diagram for Login

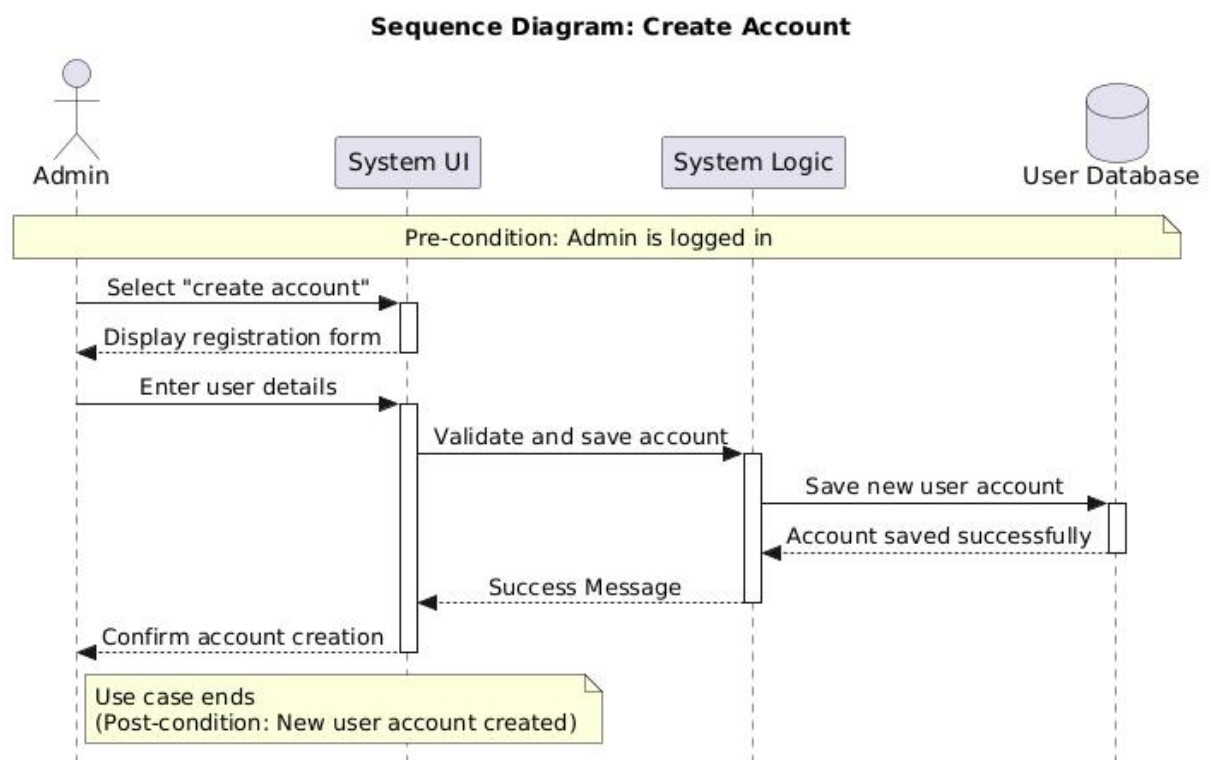


Figure 3.3: Sequence Diagram for Create Account

Use Case: Delete account

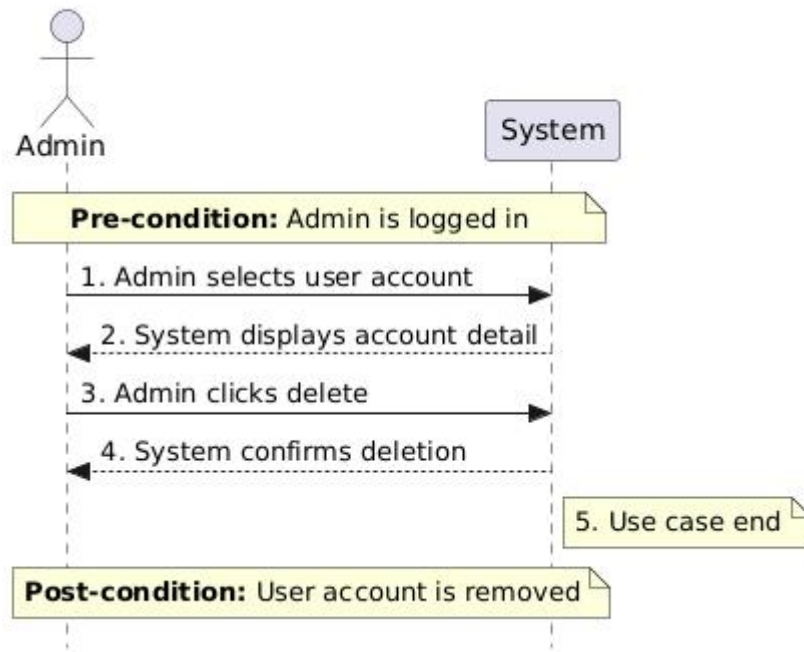


Figure 3.4: Sequence Diagram for Delete Account

Use Case: Display medication

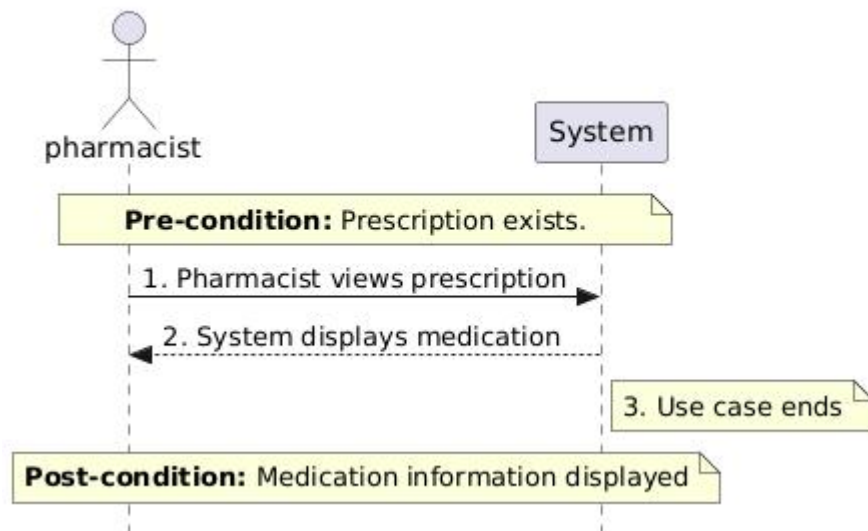


Figure 3.5: Sequence Diagram for Display Medication

Use Case: Register new patient

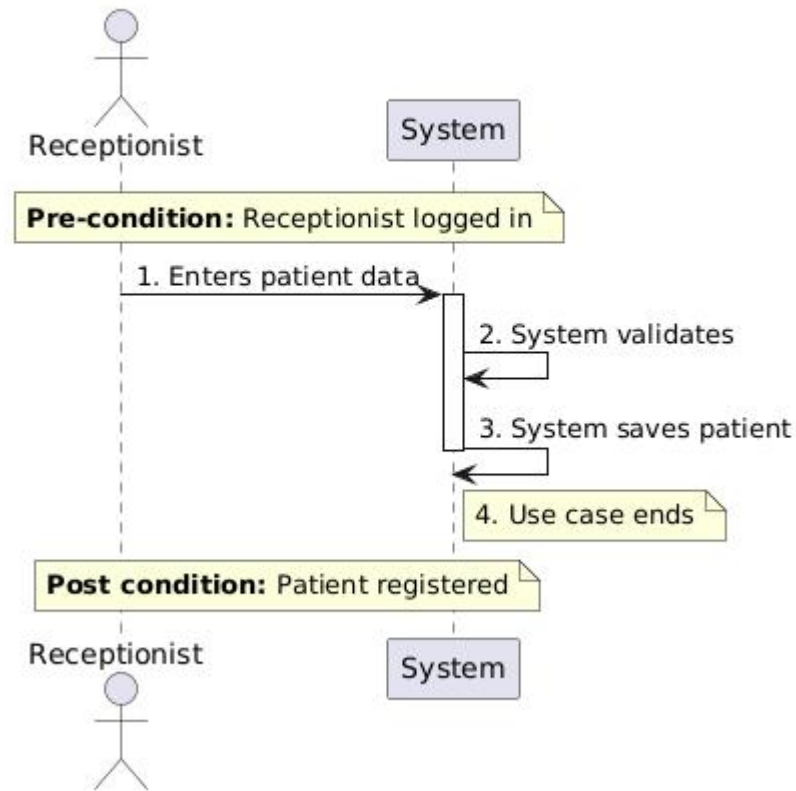


Figure 3.6: Sequence Diagram for Register New Patient

Sequence Diagram: Access Patient Record

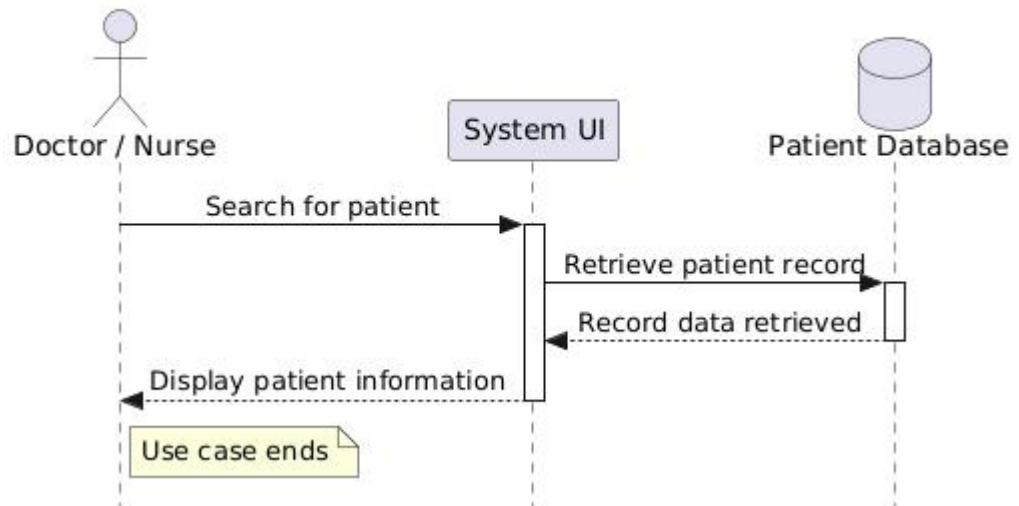


Figure 3.7: Sequence Diagram for Access Patient Record

Use Case: Request lab test

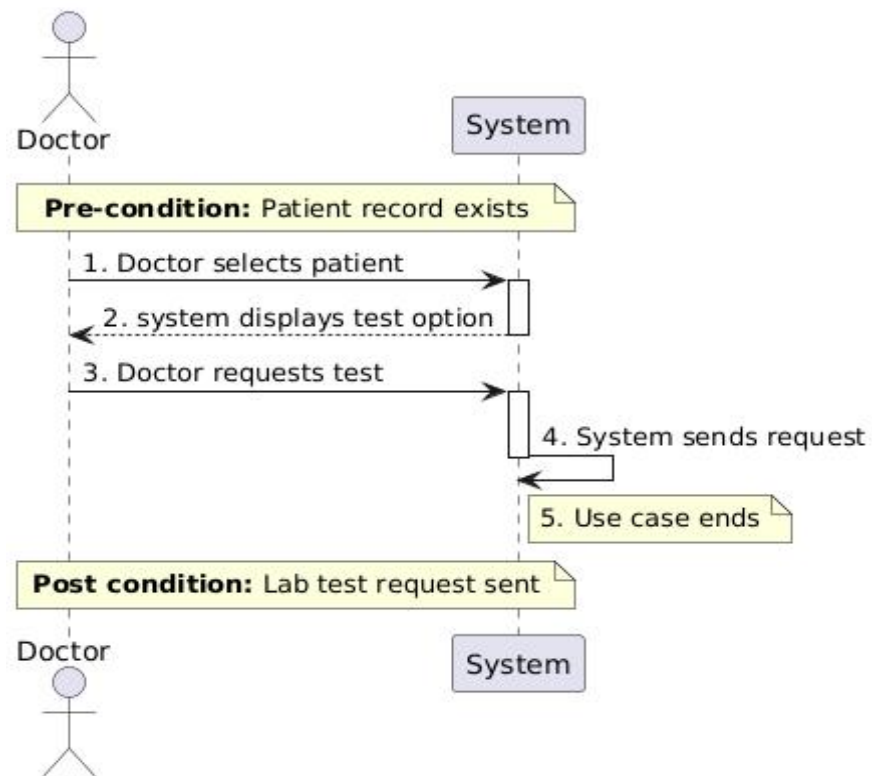


Figure 3.8: Sequence Diagram for Request Lab Test

Use Case: schedule appointment

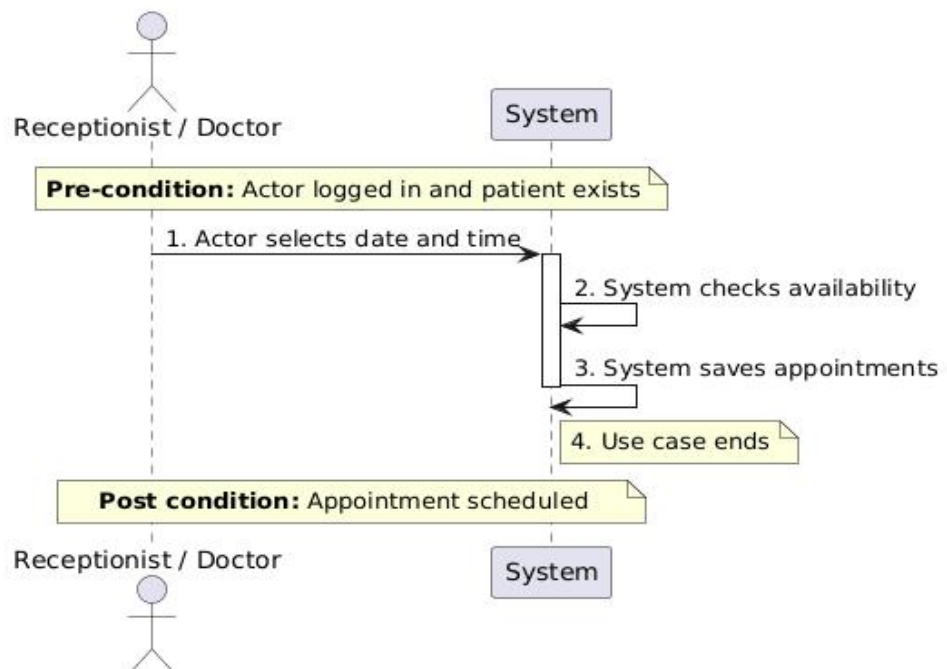


Figure 3.9: Sequence Diagram for Schedule Appointment

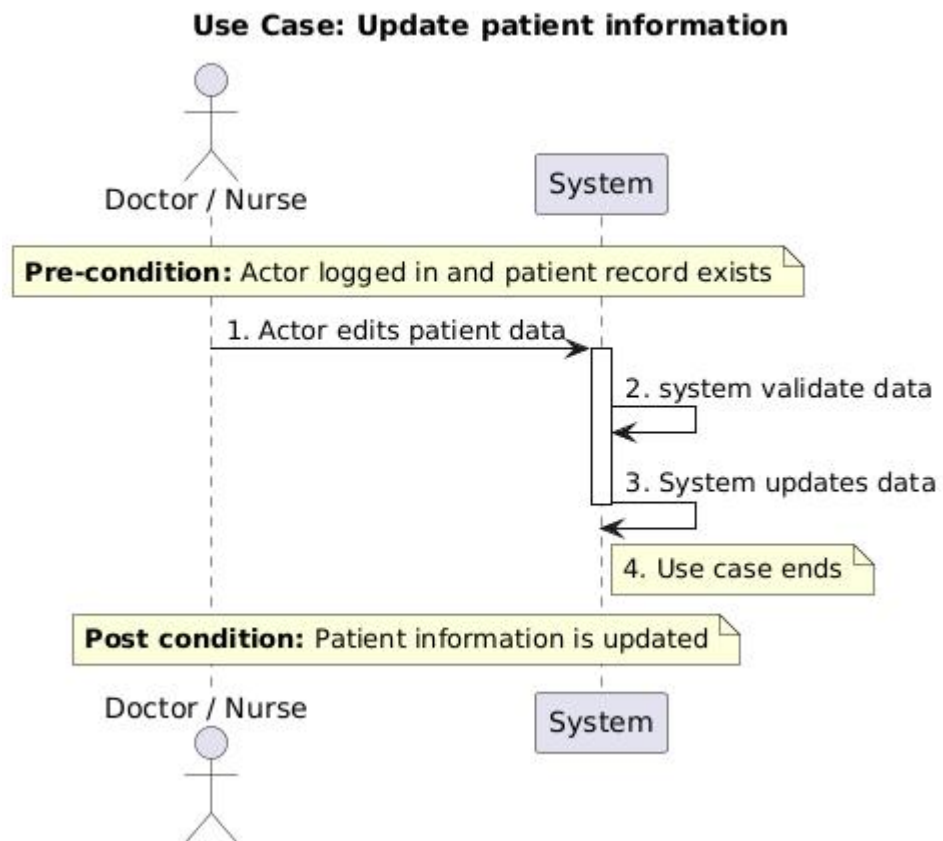


Figure 3.10: Sequence Diagram for Update Patient Information

3.4 Conceptual Class Model

Class diagrams are used to represent the structure the system in terms of objects, their attributes and nature relationship between classes. It is an abstraction of the real environment class of user and the system as a whole.

In a class diagram, classes are represented as boxes with three sections: the top section contains the class name, the middle section lists the attributes of the class, and the bottom section shows the methods (operations or behaviors). (2)

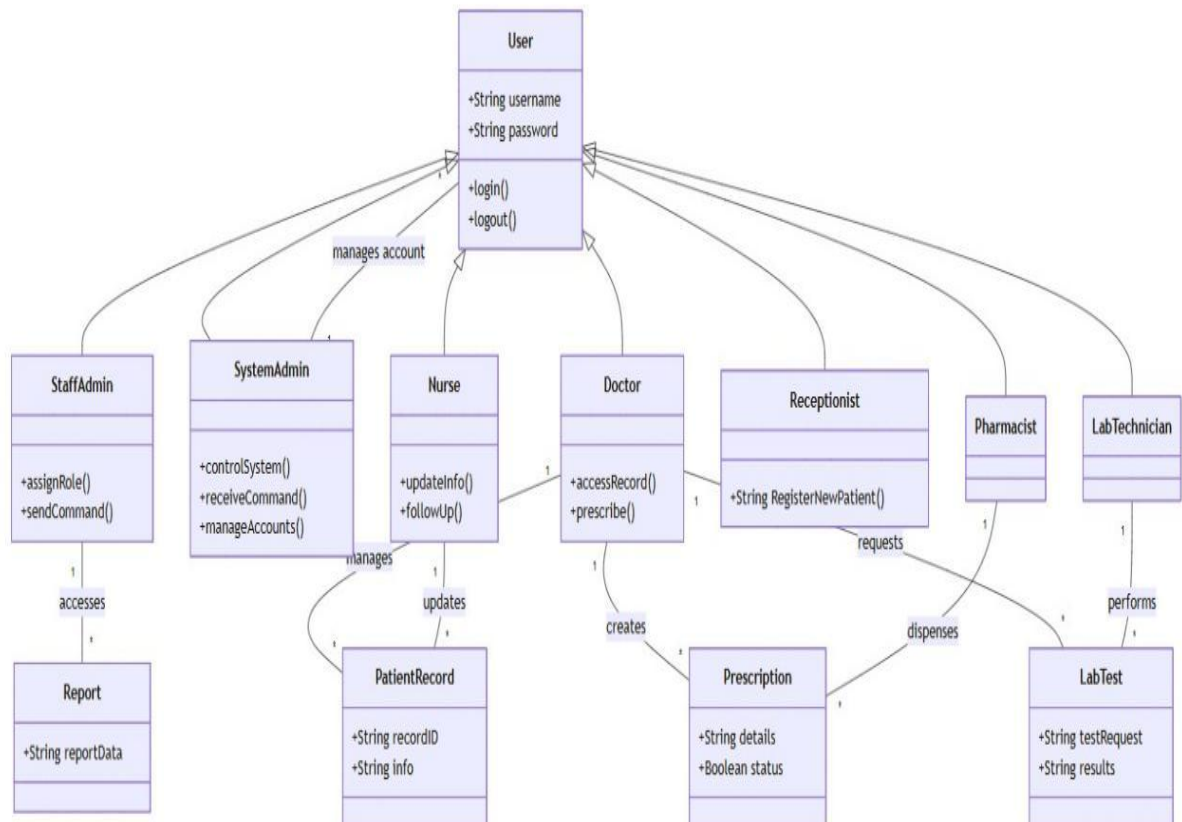


Figure 3.11: Conceptual Diagram model

3.5 Activity Diagram

Activity diagrams are graphical representations of work flows of stepwise activities and actions with support for choice. In its basic form an activity diagram is a simple and intuitive illustration of what happens in work flow, what activities can be done in parallel, and whether there are alternative paths through the work flow. (2)

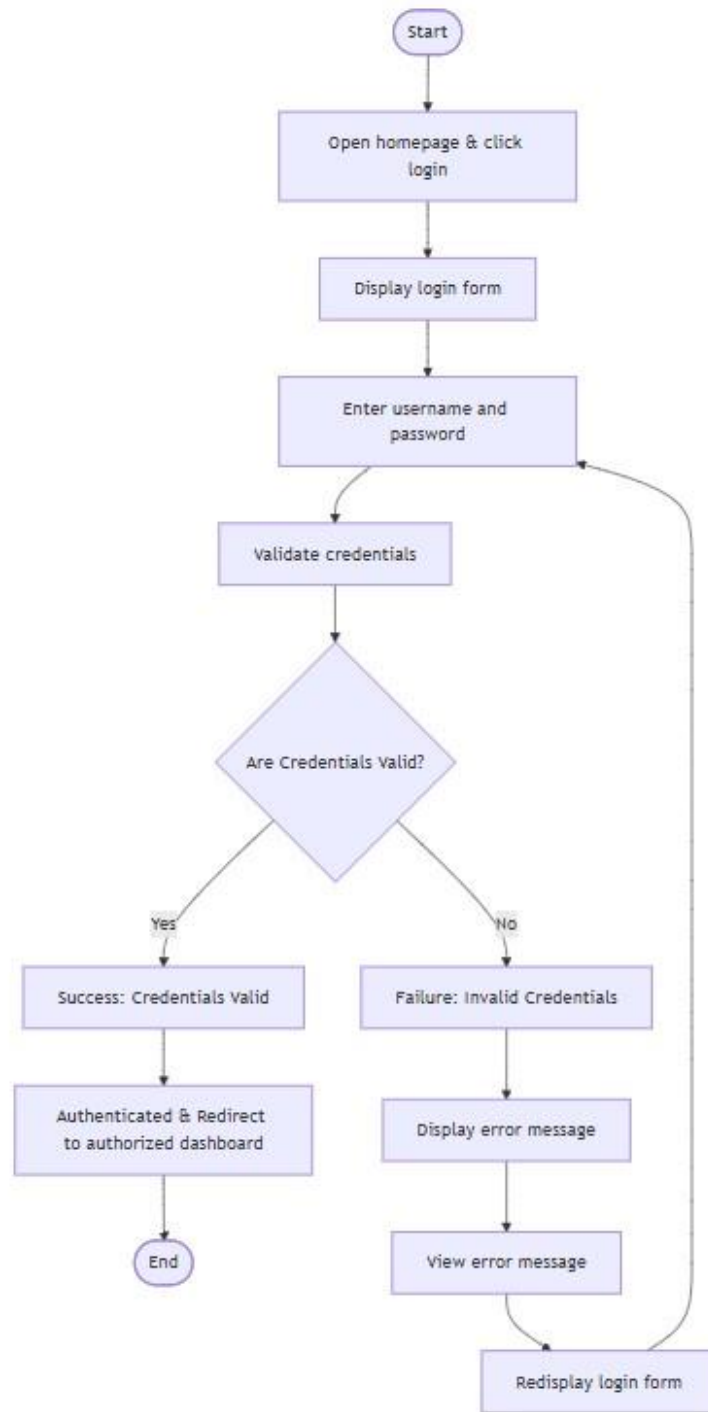


Figure 3.12: Activity Diagram for Login

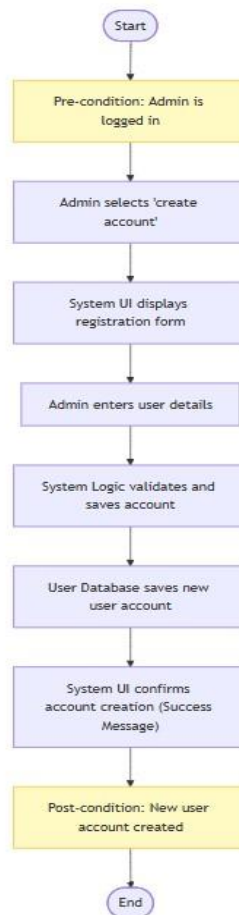


Figure 3.13: Activity Diagram for Create Account

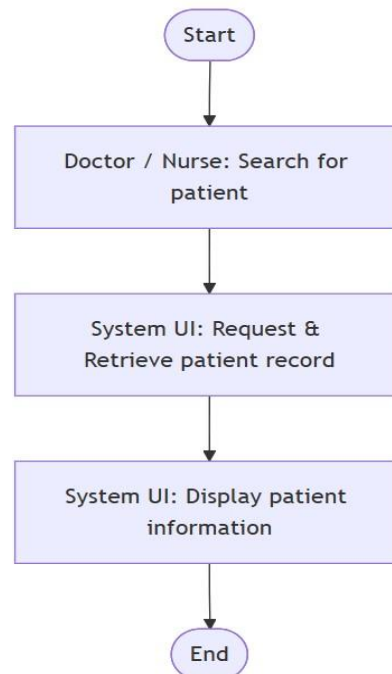


Figure 3.14: Activity Diagram for Access Patient Record

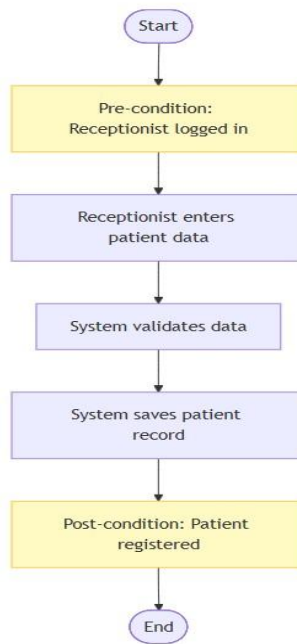


Figure 3.15: Activity Diagram for Register New Patients

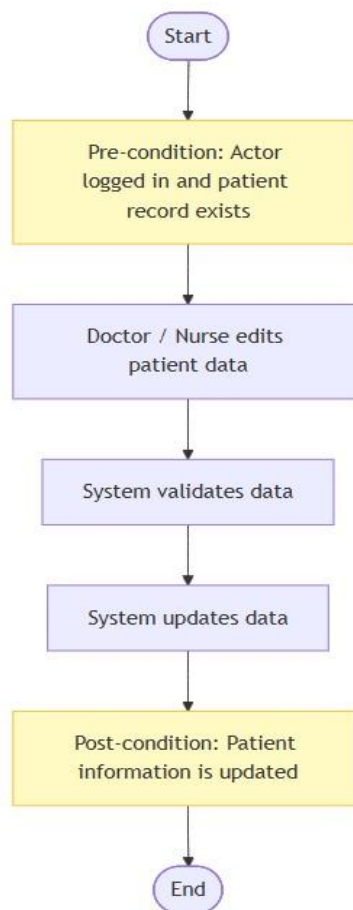


Figure 3.16: Activity Diagram for Update New Patient

CHAPTER FOUR

SYSTEM DESIGN

4.1 Introduction

This chapter presents the **system design** of the Adigrat Hospital Management System (AHMS) for Adigrat Hospital. System design is a critical phase in system development as it translates the system requirements identified in the previous chapters into a structured and technical blueprint for system implementation.

The chapter describes how the proposed system is logically and physically designed to meet the functional and non-functional requirements of the hospital. It outlines the overall system architecture, database design, system modules, user interfaces, and data flow within the system. The design ensures that the system is scalable, secure, user-friendly, and capable of supporting different hospital departments.

The system design of the AHMS adopts **web-based client-server architecture**, enabling authorized users such as doctors, nurses, laboratory technicians, pharmacists, receptionists, and system administrators to access and update patient health records through a centralized platform. Emphasis is placed on data integrity, confidentiality, and role-based access control to protect sensitive medical information.

This chapter serves as a foundation for the implementation phase by clearly defining how system components interact and how data is processed and stored within the Adigrat Hospital Management System.

4.2 Design Goal

The main goal of designing the AHMS for Adigrat Hospital is to develop a reliable, efficient, and secure system that manages patient health information electronically. The system is designed to replace the existing manual and paper-based record-keeping processes, and almost all activities of thereby improving accuracy, accessibility, and efficiency in hospital operations.

The specific design goals of the system include:

- To provide centralized patient record management the system is designed to store all patient health records in a centralized database, enabling authorized hospital staff to access accurate and up-to-date information in real time.

- To ensure data security and confidentiality the design incorporates user authentication, role-based access control, and secure data handling mechanisms to protect sensitive medical information from unauthorized access.
- To support multiple hospital departments the system is designed to serve different users such as doctors, nurses, laboratory technicians, pharmacists, receptionists, and system administrators, each with specific roles and permissions.
- To improve efficiency and reduce errors by automating data entry, retrieval, and record updates, the system minimizes human errors associated with manual record keeping and enhances workflow efficiency.
- To ensure system scalability and maintainability the system design allows future expansion, such as adding new modules, users, or departments, without major modifications to the existing system.
- To enhance usability and accessibility the system is designed with a user-friendly interface that can be accessed through standard web browsers, reducing the learning curve for hospital staff.

4.3 System Architecture

The AHMS for Adigrat Hospital is designed using **web-based client-server architecture**. This architectural design enables multiple users from different hospital departments to access and manage patient health records through a centralized system.

The system architecture is organized into **three main layers**, each responsible for specific functions within the system.

4.3.1 Presentation Layer (Client Layer)

The presentation layer provides the interface through which users interact with the system. It is accessed using standard web browsers installed on computers within the hospital network.

This layer is responsible for:

- Displaying system interfaces such as login pages, dashboards, and data entry forms
- Accepting user inputs and sending requests to the server
- Presenting processed information returned from the server

4.3.2 Application Layer (Server Layer)

The application layer handles the core logic of the system. It processes user requests received from the presentation layer and applies the defined business rules of the hospital.

This layer is responsible for

- User authentication and authorization
- Managing patient records and medical history
- Handling laboratory results, prescriptions, and reports
- Enforcing role-based access control

The application layer acts as a bridge between the user interface and the database, ensuring secure and efficient data processing.

4.3.3 Data Layer (Database Layer)

The data layer is responsible for storing and managing all system data. It contains structured databases that hold patient information, user accounts, medical records, laboratory results, and prescription data.

This layer ensures:

- Data integrity and consistency
- Secure storage of sensitive medical information
- Efficient data retrieval and updating

The separation of the data layer from other layers enhances system security and maintainability.

4.3.4 Architecture Overview

In this architecture, users interact with the system through a web browser. Requests are sent to the application server, where they are processed according to system rules. The server then communicates with the database to retrieve or update information and returns the results to the user interface.

4.4 Class Diagram

4.4.1 Introduction

The class diagram represents the **static structure** of the AHMS. It shows the main classes in the system, their attributes, methods, and the relationships among them. The class diagram helps in understanding how system objects interact and how data is organized within the system.

4.4.2 Main Classes of the System

Digital Health Record System - Classes

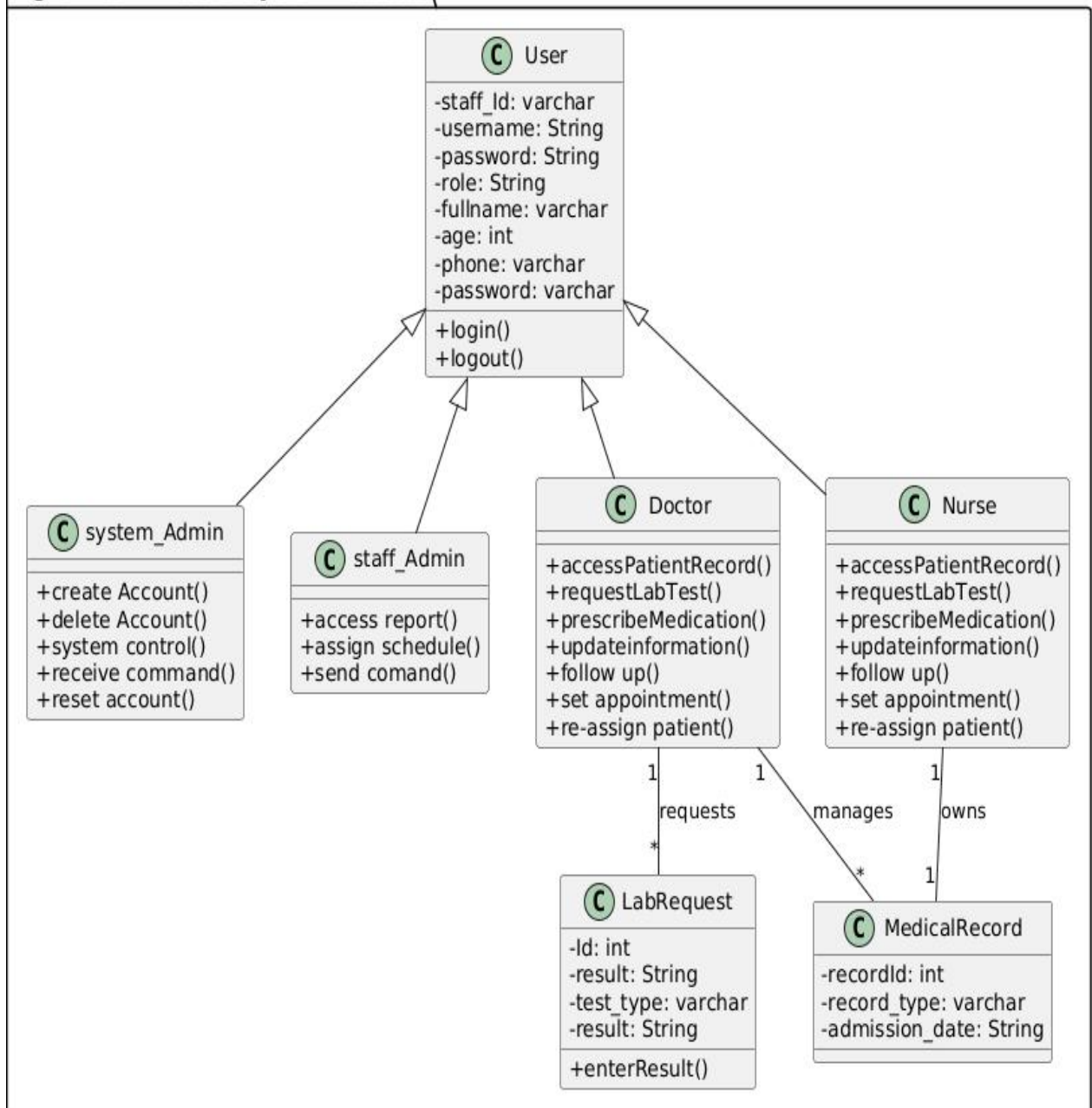


Figure 4.1: class diagram for AHMS

4.4.3 Relationships between Classes

- A User can manage many Patients
- A Patient can have multiple Medical Records
- A Patient can have multiple Laboratory Results
- A Patient can have multiple Prescriptions
- A Patient can have multiple Appointments

4.5 Deployment Diagram

A deployment diagram is used to illustrate the physical architecture of the system by showing the relationship between hardware nodes and the software components deployed on them. It describes how the system is distributed across different machines and how these machines communicate during run-time.

In the AHMS, the deployment diagram shows how hospital users access the system through client computers connected to a centralized server environment. The diagram identifies the client devices, the web/application server, and the database server as the main physical nodes of the system.

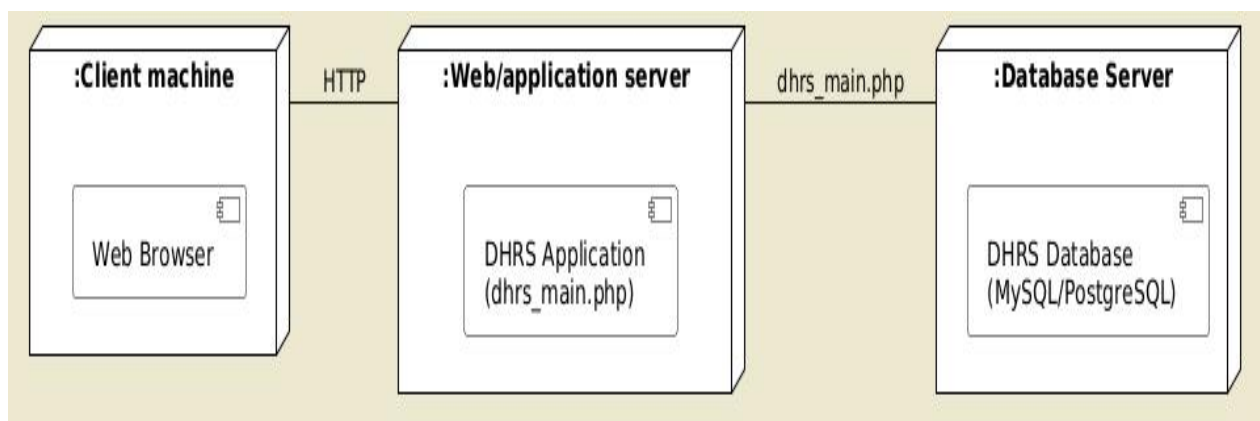


Figure 4.2: Deployment Diagram of AHMS

The deployment architecture of the AHMS consists of the following nodes:

1. Client Node (Hospital Staff Computers)

This node represents computers used by hospital staff such as doctors, nurses, laboratories technicians, pharmacists, receptionists, Staff administrators and system administrators.

Each client device accesses the system using a web browser (e.g., Chrome or Firefox). The browser sends HTTP requests to the web/application server.

2. Web/Application Server Node

This node hosts the web server (Apache) and the application logic developed using PHP.

The web/application server is responsible for:

- Processing user requests
- Performing authentication (login.php)
- Managing patient records
- Enforcing role-based access control
- Communicating with the database server

The web server acts as an intermediary between the client computers and the database.

3. Database Server Node

This node contains the MySQL database that stores all system data, including:

- User account
- Patient information
- Medical records
- Laboratory results
- Prescriptions

The application server communicates with the database server using SQL queries to retrieve and update information.

Deployment Process Description

As shown in the deployment diagram, hospital users access the AHMS through a web browser on their respective computers. The browser sends requests to the web/application server over the hospital intranet or internet using HTTP protocol.

The web/application server processes the requests using PHP scripts and communicates with the MySQL database server to store or retrieve data. After processing, the server sends the response back to the client's browser for display.

This deployment architecture ensures centralized data management, improved security, easier maintenance, and efficient access to patient records across different hospital departments.

4.6. Persistence Data Management

As discussed in the previous sections, the AHMS for Adigrat Hospital consists of several integrated modules. These include User Management System, Patient Registration System, Medical Record

Management System, Laboratory Management System, Prescription Management System, and Appointment Management System. (1)

Mapping

To store information persistently, system objects are mapped into database tables, and object attributes are mapped into table fields. Based on the identified system objects in the AHMS, major database tables are implemented in the selected DBMS (MySQL).

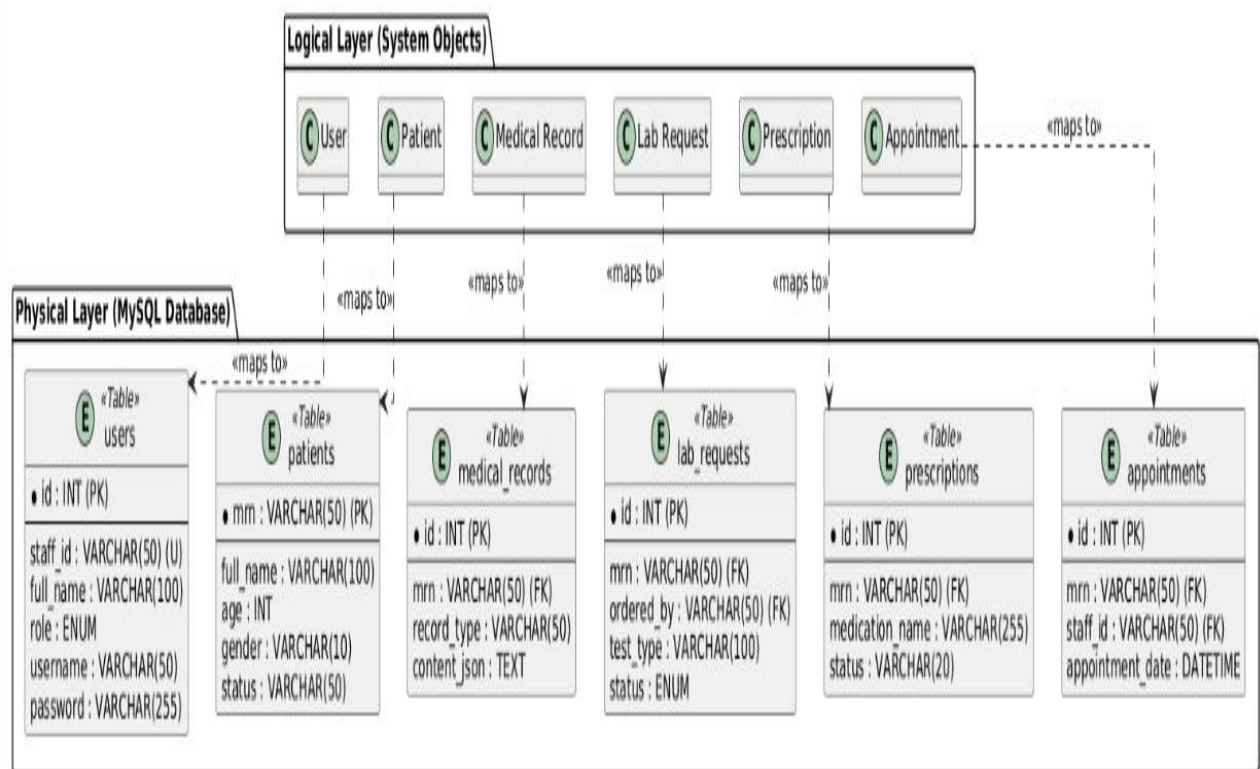


Figure 4.3: Mapping object

4.7 User Interface

User The user interface (UI) for the AHMS is designed to provide a flexible and interactive experience for hospital staff across the institution. Developed as a web-based application, the interface is engineered to be accessible on different devices equipped with a web browser, ensuring that the system is not restricted to a single desktop computer. The goal of this design is to ensure that the application remains responsive, allowing for efficient navigation and data entry regardless of the workstation being used. By prioritizing user-centered design, the interface simplifies complex medical workflows into an intuitive digital environment, enhancing the overall productivity of healthcare professionals at Adigrat

Hospital.



Figure 4.4: Home Page

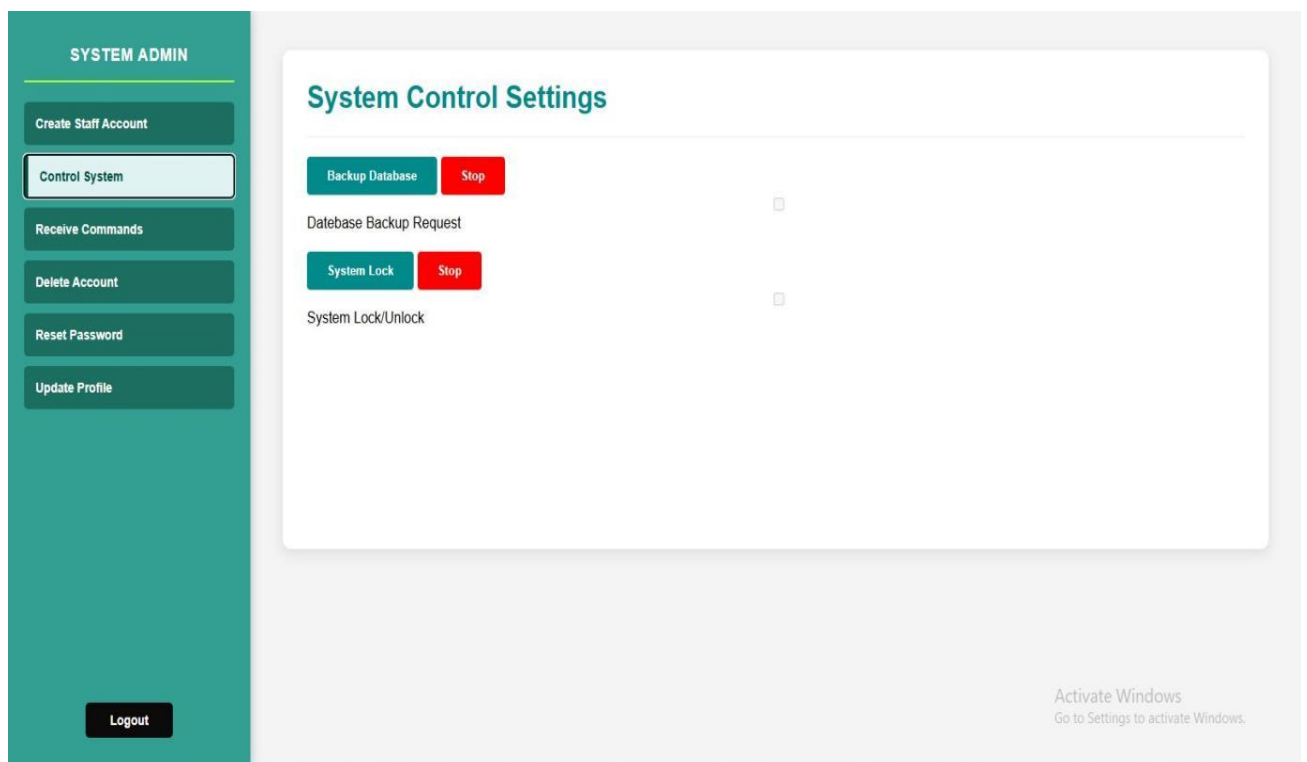


Figure 4. 5: system admin page

RECEPTION PANEL

Register New Patients

Assign patient

Schedule Appointment

Monitor Duty Staff

Update Personal Profile

Logout

Eng / አማርኛ

INDIVIDUAL FOLDER

Enter details to generate a new Hospital ID.

Medical Record Number:

MRN_000002

Full Name:

Enter patient name

Date of Registration:

2026-05-04

Date of Birth:

yyyy-mm-dd

Gender:

Male

Nationality:

Ethiopian

Region:

Tigray

Wereda/SubCity:

Kebele:

Ketena/Gott:

House Number:

Phone Number (Optional):

10-15 digits

Register

Activate Windows
Go to Settings to activate Windows.

Figure 4. 6: receptionist page

CHAPTER FIVE

IMPLEMENTATION AND TESTING

5.1 Implementation Overview

Implementation in the system includes implementing the attributes and methods of each object and integrating all the objects in the system, to function as a single system. The implementation activity spans the gap between the detailed object design model and a complete set of source code files that can be compiled together.

In this chapter we mainly focus on the implementation part, implementation concerned with the type of material (Hardware and Software required), techniques to develop the system, algorithm for the system, code samples of the system, data preparation, how to install the system, some testing techniques, startup strategy for the new installed system and some others are briefly described. (1)

5.1.1 Objective of the implementation

The objective of systems implementation phase is to convert the final physical system Specifications into working and reliable system, document the work that has been done, and provide help for current and future users and caretakers of the system.

5.2 Sample code and overview

Sample code for login

```
<body>
<header>
  <div class="header-brand">
    
    <div class="brand-text">
      <h1>ADIGRAT GENERAL HOSPITAL</h1>
      <h1>ሓፈሽ ዋ ሆስፒታል ግዳግራት</h1>
    </div>
  </div>
</header>

<nav>

  <div class="nav-inner">
    <a href="#" class="nav-login-btn" onclick="openLogin(); return
false;">LOGIN</a>
```

```

</div>
</nav>

<section class="hero-slider">
  <div class="slide active">
    <div class="slide-overlay">
      <div class="slide-text">
        <h2>Excellence in Education, Academics, Research and Community
Engagement</h2>
      </div>
    </div>
  </div>
  <div class="slide">
    <div class="slide-overlay">
      <div class="slide-text">
        <h2>Modern Healthcare Data Management for Better Patient Care</h2>
      </div>
    </div>
  </div>
  <div class="slide">
    <div class="slide-overlay">
      <div class="slide-text">
        <h2>A Vibrant Learning Environment</h2>
      </div>
    </div>
  </div>

  <!-- SLIDER ARROWS RETURNED -->
  <button class="slider-prev" onclick="changeSlide(-1)"><i class="fas fa-chevron-
left"></i></button>
  <button class="slider-next" onclick="changeSlide(1)"><i class="fas fa-chevron-
right"></i></button>
</section>

<footer>
  <p>2026 Adigrat General Hospital</p>
</footer>

<div class="login-overlay" id="loginOverlay">
  <div class="login-container">
    <span class="close-btn" onclick="closeLogin()">&times;</span>
    <h3>AHMS Staff</h3>
    <div class="input-group"><input type="text" id="username"
placeholder="Username"></div>

```

```

    <div class="input-group" style="position: relative;">
    <input type="password" id="password" placeholder="Password">

    <!-- eye sign (Emoji) -->
    <span id="eyeIcon" onclick="togglePass()"
        style="position: absolute; right: 15px; top: 50%; transform: translateY(-50%);
        cursor: pointer; font-size: 1.2rem; user-select: none;">
    </span>
    </div>
    <button class="submit-btn" onclick="login()">Login</button>
    <p id="msg" style="color:red; text-align:center; margin-top:10px;"></p>
    </div>
</div>

<script>
    // to unseen the password if default back button clicked
    window.history.forward();
    function noBack() { window.history.forward(); }

    window.onpageshow = function(event) {
        document.getElementById("username").value = "";
        document.getElementById("password").value = "";
        document.getElementById("msg").innerText = "";
    };
    function openLogin()
    { document.getElementById("loginOverlay").classList.add("open"); }
    function closeLogin()
    { document.getElementById("loginOverlay").classList.remove("open"); }
    let current = 0;
    const slides = document.querySelectorAll(".slide");
    function changeSlide(dir) {
        slides[current].classList.remove("active");
        current = (current + dir + slides.length) % slides.length;
        slides[current].classList.add("active");
    }
    // Auto slide
    let autoSlide = setInterval(() => changeSlide(1), 8000);
    // Stop auto slide on manual click
    document.querySelectorAll('.slider-prev, .slider-next').forEach(btn => {
        btn.addEventListener('click', () => clearInterval(autoSlide));
    });
    function login() {

```

```

const u = document.getElementById("username").value;
const p = document.getElementById("password").value;
const m = document.getElementById("msg");
if(!u || !p) { m.innerText = "Please enter credentials"; return; }
const fd = new FormData();
fd.append("username", u); fd.append("password", p);
fetch("login_process.php", { method: "POST", body: fd })
    .then(r => r.json())
    .then(d => {
        if (d.status === "success") window.location.href = d.redirect;
        else m.innerText = d.message;
    }).catch(() => m.innerText = "Connection error!");
}
document.addEventListener("keydown", function(event) {
    const overlay = document.getElementById("loginOverlay");

    // Enter ↵
    if (overlay.classList.contains("open")) {
        if (event.key === "Enter") {
            event.preventDefault();
            login();
        }
    }
});

// to show and hidden password
function togglePass() {
    const passInput = document.getElementById('password');
    const eyeIcon = document.getElementById('eyeIcon');
    if (passInput.type === 'password') {
        passInput.type = 'text';
        eyeIcon.innerText = ' ';
    } else {
        passInput.type = 'password';
        eyeIcon.innerText = ' ';
    }
}
</script>
</body>
</html>

```


Sample code for register patient

```
// PATIENT REGISTRATION (Individual Folder)
if(action === 'register') {
    area.innerHTML = `
        <button class="lang-switch" id="lang-btn" onclick="toggleLanguage()">Eng /
አ ማ</button>
        <h1 id="t-folder">INDIVIDUAL FOLDER</h1>
        <p id="t-desc">Enter details to generate a new Hospital ID.</p>
        <div style="display:flex; gap:20px;">
            <div class="form-group" style="flex:0.33;">
                <label id="f-mrn">Medical Record Number:</label>
                <input type="text" id="p_mrn" readonly style="background:#eee;">
            </div>
            <div class="form-group" style="flex:1;"><label id="f-name">Full
Name:</label><input type="text" id="p_name" placeholder="Enter patient
name"></div>
        </div>
        <div style="display:flex; gap:10px;">
            <div class="form-group" style="flex:1;">
                <label id="f-reg-date">Date of Registration:</label>
                <input type="Date" id="p_reg_date" readonly style="background:#eee;
cursor:not-allowed;">
            </div>
            <div class="form-group" style="flex:1;"><label id="f-dob">Date of
Birth:</label><input type="Date" id="p_dob" max="<?php echo date('Y-m-d'); ?>"
onchange="calculateAge()"></div>
            <div class="form-group" style="flex:0.5;"><label id="f-
gender">Gender:</label><select
id="p_gender"><option>Male</option><option>Female</option><option>Other</o
ption></select></div>
        </div>
        <div style="display:flex; gap:20px;">
            <div class="form-group" style="flex:1;">
                <label id="f-nat">Nationality:</label>
                <select id="p_nationality" onchange="toggleOtherField(this,
'nat_other_div')">
                    <option>Ethiopian</option><option
value="Other">Other</option>
                </select>
            </div>
        </div>
    `;
}
```

```

        <div id="nat_other_div" style="display:none; margin-top:10px;"><input type="text" id="p_nat_other" placeholder="Specify nationality"></div>
    </div>
    <div class="form-group" style="flex:1;">
        <label id="l-region">Region:</label>
        <select id="p_region" onchange="toggleOtherField(this, 'reg_other_div')">
            <option>Tigray</option><option>Afar</option><option>Amhara</option><option>Oromia</option><option>Addis Ababa</option>
            <option>DireDawa</option><option>Sidama</option><option>Harar</option><option>Somalia</option>
            <option>Benishangul Gumuz</option><option>South Ethiopia</option><option value="Other">Other</option>
        </select>
        <div id="reg_other_div" style="display:none; margin-top:10px;"><input type="text" id="p_reg_other" placeholder="Specify region"></div>
    </div>
</div>
<div style="display:flex; gap:20px;">
    <div class="form-group" style="flex:1;"><label id="l-wereda">Wereda/SubCity:</label><input type="text" id="p_wereda"></div>
    <div class="form-group" style="flex:1;"><label id="l-kebele">Kebele:</label><input type="text" id="p_kebele"></div>
    <div class="form-group" style="flex:1;"><label id="l-ketena">Ketena/Gott:</label><input type="text" id="p_ketena"></div>
</div>
<div style="display:flex; gap:20px;">
    <div class="form-group" style="flex:1;"><label id="l-house">House Number:</label><input type="text" id="p_house"></div>
    <div class="form-group" style="flex:1;">
        <label id="l-phone">Phone Number (Optional):</label>
        <input type="text" id="p_number" oninput="this.value = this.value.replace(/^[^0-9+]/g, '')" maxLength="15" placeholder="10-15 digits">
    </div>
</div>
<button class="action-btn" id="reg-btn-text" onclick="processRegistration()">Register</button>
<p id="reg-msg"></p> `;

```

```

    const today = new Date();
    const yyyy = today.getFullYear();
    const mm = String(today.getMonth() + 1).padStart(2, '0');
    const dd = String(today.getDate()).padStart(2, '0');
    const formattedToday = `${yyyy}-${mm}-${dd}`;
    const regDateInput = document.getElementById('p_reg_date');
    regDateInput.value = formattedToday;
    regDateInput.max = formattedToday;
    document.getElementById('p_reg_date').valueAsDate = new Date();
    fetch('get_next_mrn.php').then(res => res.text()).then(data =>
    { document.getElementById('p_mrn').value = data.trim(); });
    }

```

5.3 Testing Overview

Testing is the process of executing a system with the objective of identifying errors and ensuring that it meets specified requirements. Testing enhances the reliability and integrity of the AHMS by detecting defects in system design, implementation, and functionality. It helps in identifying error-prone areas and ensures that the system operates according to user requirements. In this phase, the entire AHMS was tested to verify that all modules function correctly and meet both functional and non-functional requirements. The main reference document for this process was the system requirements specification. The primary goal was to confirm that the system satisfies hospital operational needs. (1)

5.3.1 Testing by Scope

5.3.1.1 Unit Testing

Unit testing is performed to verify that individual modules of the system function correctly before integration. Each module of the AHMS was tested independently to ensure that it produces the expected output.

The following modules were tested individually

- User Authentication Module
- Patient Registration Module
- Medical Record Module
- Laboratory Result Module
- Prescription Module
- Appointment Module

Table 5.1: Login Module

Test Case Name		Login Authentication	
Purpose		To verify that only authorized users can access the system	
Input	Expected Result	Actual Output	Pass/Fail
Valid username and valid password	System successfully logs in and displays user dashboard	Dashboard displayed	Pass
Valid username and invalid password	System displays error message	Error message displayed	Fail
Invalid username and valid password	System displays error message	Error message displayed	Fail
Invalid username and invalid password	System denies access	Access denied	Fail
Null username and password	System prompts user to fill required fields	Validation message shown	Fail

5.3.1.2 Integration Testing

Integration testing was conducted after combining all individual modules of the AHMS. The purpose was to verify that modules interact correctly with each other and with the database.

The following integrations were tested:

- Login module integrated with User database
- Patient registration integrated with Medical Record module
- Laboratory results linked with patient records
- Prescription module integrated with doctor and patient records
- Appointment module linked with department and patient information
- The system was tested to ensure that data flows correctly between modules without errors.

5.3.2 Testing by Requirements (System Testing)

System testing was performed to verify that the entire AHMS meets both functional and non-functional requirements. This testing evaluated the system as a complete integrated unit.

Functional Requirements Testing

The system was tested to ensure:

- Users can log in based on role

- Patients can be registered successfully
- Medical records can be added and retrieved
- Laboratory results can be stored and viewed
- Prescriptions can be recorded
- Appointments can be scheduled
- Detail history can be viewed

All functional requirements were verified successfully.

Non-Functional Requirements Testing

Performance

The system was tested to ensure that it responds quickly to user requests. The AHMS performs efficiently under normal hospital usage conditions.

Volume

To ensure the system could handle the actual demands of Adigrat Hospital, scalability and stress testing were conducted using simulated datasets. The database demonstrated the ability to store and retrieve large volumes of medical records with high performance and zero data loss, confirming its readiness for practical implementation.

Documentation

Comprehensive user documentation was developed and reviewed to provide a step-by-step guide for hospital staff on navigating the entire system workflow. This includes specific modules for administrators, doctors, nurses, receptionists, pharmacists, and laboratory technicians. The documentation covers the complete lifecycle of patient management—from initial registration and triage to laboratory testing, diagnosis, prescription fulfillment, and final report generation—ensuring each user group can operate their respective modules efficiently and accurately.

Robustness

The system underwent rigorous robustness testing to ensure stability throughout the entire clinical process. This included stress-testing the system with invalid input conditions, such as incorrect data formats or out-of-range values, across all modules. The AHMS successfully demonstrated its ability to maintain data integrity and prevent system crashes by validating inputs at every stage. It provides clear, context-specific error messages that guide users to correct mistakes immediately, ensuring the overall process remains uninterrupted even when user errors occur.

5.3.3 Test Implementation (Validation Testing)

The following validation mechanisms were implemented:

Password Validation

If an unauthorized user attempts to log in, the system displays the message:

“Invalid username or password. Please try again.”

Data Type Validation

The system accepts only valid data types (e.g., numeric values for phone numbers and IDs). If invalid data is entered, the system displays an appropriate validation message.

Input Validation and Security

The system implements strict input validation to maintain data integrity and security across all hospital operations. Mandatory fields—such as patient name, diagnosis, and appointment date—cannot be left empty; if a required field is blank, the AHMS prompts the user to provide the missing information before proceeding.

Furthermore, the system enforces a strong password policy to protect sensitive medical records. During registration or password updates, the system validates that passwords meet specific complexity requirements, including a minimum length of 8 characters and a mandatory combination of uppercase letters, lowercase letters, numerical digits, and special characters. This comprehensive validation process ensures that the AHMS for Adigrat Hospital is not only reliable and user-friendly but also secure against unauthorized access, effectively supporting the hospital's clinical and administrative workflows.

5.4 Hardware and Software Acquisition

For the implementation of the AHMS for Adigrat Hospital, the following hardware and software components were used.

Hardware Requirements

The hardware components required for the development and deployment of the AHMS include:

- **Computers:** Used by hospital staff (doctors, nurses, receptionists, laboratory technicians, pharmacists, staff administrators and system administrators) to access the system.
- **Server Computer:** Hosts the web/application server and database server.
- **Printer:** Used for printing patient reports, schedules, and documentation.
- **Network Devices (Router/Switch):** For establishing hospital local area network (LAN).
- **RJ-45 Network Cables:** For connecting computers within the network.

- **External Storage Device (Flash Disk/External Drive):** For data backup and file transfer.

Software Requirements

The following software tools were used in the development and deployment of the AHMS:

- Operating System: Windows 10 /pro
- Programming Languages: PHP, JavaScript, HTML, and CSS,
- Server Software: XAMPP (Apache Web Server)
- Database Management System: MySQL
- Modeling Tools: Microsoft Visio, Draw.io (for UML diagrams)
- Web Browser: Google Chrome, Firefox
- Documentation Tools: Microsoft Word, WPS office
- Presentation Tool: Microsoft PowerPoint

These tools were selected due to their reliability, ease of use, and compatibility with web-based applications.

5.5 User Manual Preparation

To access the AHMS, each hospital staff member must have an authorized account created by the System Administrator.

The system administrator is responsible for:

- Creating user accounts
- Receive commands from staff admin
- Control system

Users must log into the system using their assigned **username** and **password**. After successful authentication, users are redirected to their respective dashboards based on their roles.

The user manual includes:

- Steps for logging into the system
- Instructions for registering patients
- Guidelines for adding medical records
- Procedures for entering laboratory results
- Instructions for generating prescriptions and reports

The manual serves as a guide to help hospital staff operate the system efficiently.

5.6 Installation Process

Since the AHMS is a web-based system, it does not require installation on every client computer. The system is hosted on a central server within the hospital network.

The installation process involves:

- Installing XAMPP on the server machine.
- Configuring Apache and MySQL services.
- Importing the AHMS database into MySQL.
- Placing the system project files in the server's root directory.
- Configuring network settings for access through LAN.

Once the server setup is complete, users can access the system through a web browser using the server's IP address or domain name.

5.7 Start-up Strategy

After deployment, authorized hospital staff can access the AHMS through the hospital network.

The start-up procedure includes:

- Starting the Apache and MySQL services on the server.
- Opening a web browser on a client computer.
- Entering the system URL (e.g., <http://localhost/AHMS> or server IP address).
- Logging in using valid username and password.
- Accessing system modules according to assigned roles.

Only authorized users can access system functionalities, ensuring data security and controlled access to patient health information.

CHAPTER SIX

CONCLUSIONS AND RECOMMENDATIONS

6.1 Conclusion

The main objective of this project was to design and develop a AHMS for Adigrat Hospital in order to modernize and computerize the management of patient health information.

The developed system successfully meets the fundamental requirements of the hospital. It provides an efficient platform for managing patient registration, medical records, laboratory results, prescriptions, and user administration. By replacing the traditional manual hospital management system, the AHMS significantly reduces paperwork, minimizes data redundancy, and improves the accuracy and accessibility of patient information.

The system benefits various hospital staff members including doctors, nurses, receptionists, laboratory technicians, pharmacists, and administrators. It automates major processes within the hospital, thereby reducing workload, saving time, and improving service delivery.

Security was considered one of the primary concerns during development. The system implements user authentication and role-based access control to ensure that sensitive patient data is protected. Additionally, the system is designed to be flexible and scalable, allowing future enhancements and modifications when necessary.

Every effort has been made to ensure that the system is user-friendly, reliable, and capable of addressing the existing problems of manual hospital management at Adigrat Hospital.

6.2 Recommendations

In today's rapidly evolving technological environment, healthcare institutions must adopt modern information systems to improve efficiency, accuracy, and competitiveness. Implementing computerized systems such as the AHMS is essential for delivering quality healthcare services.

Since Adigrat Hospital currently relies heavily on manual processes in some areas, the implementation of AHMS will greatly enhance operational efficiency, reduce errors, and improve patient care services.

Based on the development and evaluation of the system, the following recommendations are made:

- Proper attention should be given during the implementation phase to ensure smooth transition from manual to digital system.
- Adequate training should be provided to all hospital staff to ensure effective utilization of the system.
- A reliable and secure network infrastructure should be established to connect all departments within the hospital.
- The system should be hosted on a secure and well-maintained server environment.
- Regular data backup mechanisms should be implemented to prevent data loss.
- Continuous system maintenance and technical support should be provided.
- The hospital management should allocate sufficient budget for system upgrades, cyber security measures, and IT infrastructure improvements.

Finally, the project team recommends that the university provide improved laboratory facilities, stable internet access, and sufficient project development time for graduating students. During the development of this project, some challenges were encountered due to limited resources and infrastructure.

References

1. **Zerue Tlahun, Mhret Aregay, Yebahrnesh Tajebe, Tsge Demewoze, Wegene Maru, Bizen G/meskel.** DEPARTMENT OF COMPUTER SCIENCE Project on Web Based Class Scheduling System for ADU. *Scribd*. [Online] Scribd, Inc. (or just Scribd), May 2017.
https://www.scribd.com/document/513030061/DEPARTMENT-OF-COMPUTER-SCIENCE-Project-o?utm_source.
2. **Yirga Haile, Meron T/brhan, Tsegay Mehari, Netsanet Gebru, G/haweria G/slassie, Hidat Teklay.** *Clinic Management System for Tekli Siwuat Adigrat Health Center*. Department of Computer Science, Adigrat University. Adigrat : s.n., 2024.

Appendix

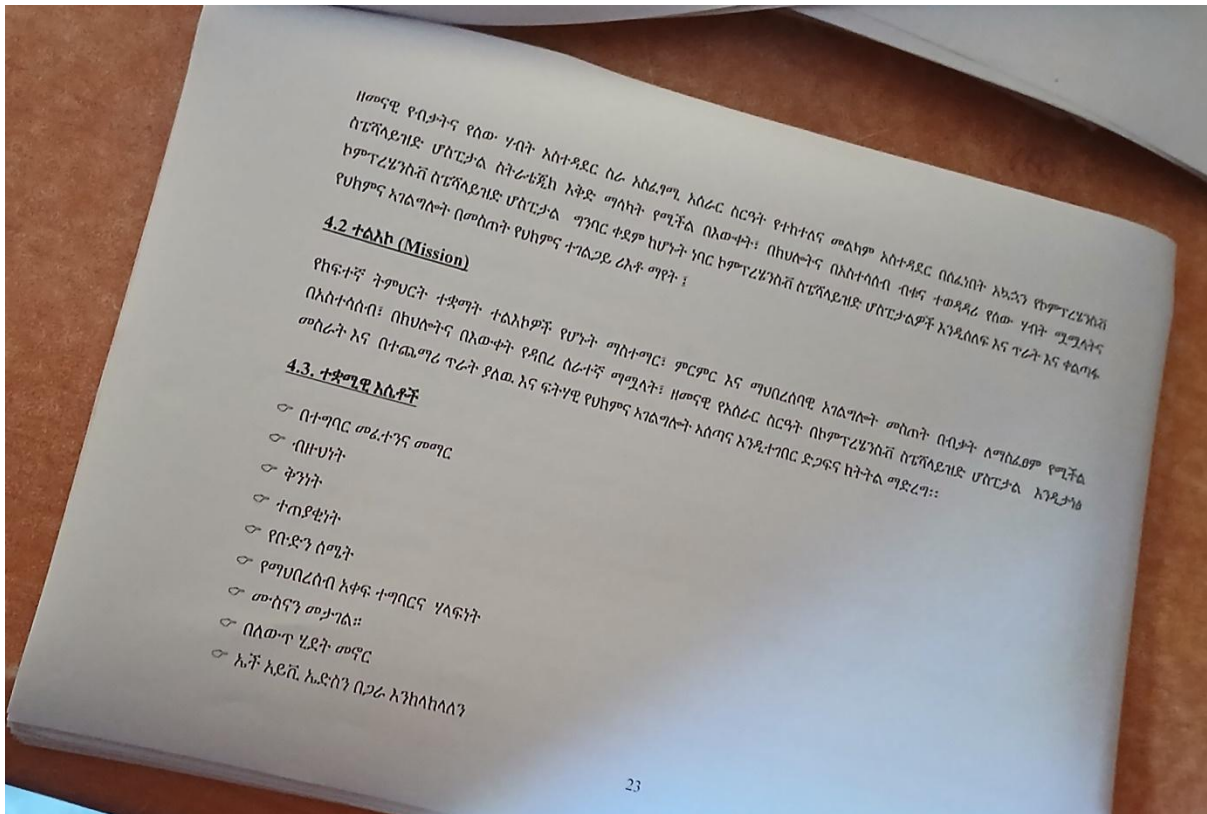


Figure 6.1: Vision and mission of the hospital

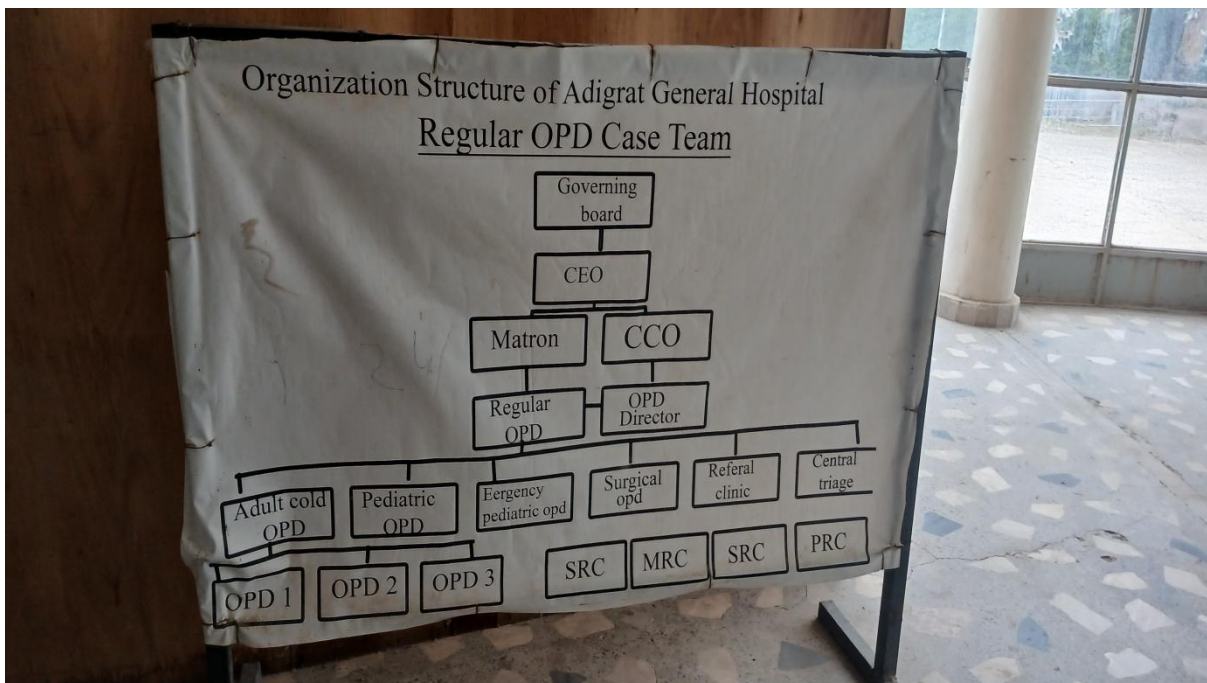


Figure 6.2: Organization structure



Figure 6.3: Receptionist room of the hospital